Name: TROY PINKNEY
Matriculation number: S1979799
BEng Honours Project Report
Trees versus Neural Networks on Tabular Data
2 May 2024

Mission Statement

Project Title: Trees versus Neural Networks on Tabular Data
Student: Troy Pinkney; s1979799
Academic Supervisor: Dr Elliot Crowley

Project Definition

The aim of this project is to gather high quality tabular datasets from the internet to allow for the comparison of the performance of deep neural networks (DNNs) with tree-based models. It is expected that trees will outperform DNNs on this type of data [1]. As a result, another objective is to understand what types of tabular data DNNs match the performance of trees and to implement a state-of-the-art regularization technique [2] that claims to be able to have DNNs outperform trees on tabular data. Regularization is a useful technique in machine learning (ML) as it counters overfitting which some models are prone too.

Main Tasks

- Find high quality tabular datasets from the internet(mainly Kaggle) that are diverse enough to produce fair benchmarks for the comparison of trees vs DNNs
- Train and test the models in order to complete the benchmarks
- Explore regularization techniques that claim to have DNNs outperform trees

Scope for Extension

- Explore recently developed state-of-the-art ML models like TabFPN [3] that are fundamentally quite different than a typical DNN
- Explore hyperparameter optimization techniques on the ML models in the benchmarks

Background Knowledge

- Tree-based models
- Deep Neural Networks
- Libraries/Frameworks so you don't have to implement the ML models from scratch such as PyTorch, Scikit-Learn and XGBoost

Resources

- My Laptop
- Google collab for GPU support

Location/s

- Meetings with my supervisor at the Alexander Graham Bell Building
- Anywhere as I only need my laptop

References

- [1] https://arxiv.org/abs/2207.08815
- [2] https://arxiv.org/abs/2106.11189
- [3] https://arxiv.org/abs/2207.01848

Abstract

The aims of this project were to compare deep-neural networks with trees on tabular data of which it is expected that trees perform better. The first step in this project was to collect several binary classification datasets from popular data science websites like Kaggle and UCI. These datasets were appropriately pre-processed adhering to good machine learning practice. This project benchmarked the logistic regression, decision tree, random forest, histogram gradient boosting, gradient boosting, XGBoost and MLP model. They were implemented using the scikit-learn, XGBoost and PyTorch libraries. This project benchmarked the default hyper-parameter models. This project explored hyper-parameter optimization techniques such as grid search, random search and Bayesian optimization. Where grid search was used to plot the hyper-parameter search spaces. Random search was used to tune an average amount of hyper-parameters. Bayesian optimization tuned a larger amount of hyper-parameters. The results from these benchmarks show that trees are marginally better than DNNs and that hyper-parameter tuning decreases the superiority gap between trees and DNNs. This project validates the claims the authors of an innovative deep-learning transformer called TabPFN made. That claims to achieve higher accuracy than the 'king of tabular data' XGBoost in a fraction of the time.

Declaration of Originality

I declare that this thesis is my
original work except where stated.

Troy Pinkney

Statement of Achievement

I started this project by researching the fundamentals of machine learning where I primarily focusing on deep-learning and trees. I then collected 44 binary classification datasets and appropriately pre-processed them. Using those datasets I benchmarked various default hyperparameter models. Unfortunately, I was unable to complete the last main task of the mission statement to 'Explore regularization techniques that make DNNs outperform trees' as I was unable to get the code accompanying the paper to work after trying it on Windows,Linux and Google Colab. I suspect it no longer works with the current version of auto-PyTorch, as was flagged as an issue on GitHub. I completed both the extensions of the project by expanding my research to hyper-parameter optimization techniques such as grid search, random search and Bayesian optimization allowing for more in depth benchmarking of the models. As well as exploring a state-of-the-art transformer, TabPFN.

Contents

List of Figures

List of Tables

Glossary

amortized Spreading out a cost over time..

API application programming interface.

bias Defines how well a function is fit to a dataset. It is typically minimized in order to find optimal ML model parameters. .

confusion matrix 2 by 2 matrix. With values for the number of true/false positives/negatives..

DNN deep neural network.

FPR false positive rate.

HPO hyper-parameter optimization.

IDE integrated development environment.

linearly separable If a dataset can be perfectly separated with a hyperplane..

ML machine learning.

MLP mulilayer perceptron.

runtime Sum of training time and time to predict(inference)..

score Measures how accurate a model is..

SGD stochastic gradient descent.

Shannon Entropy A measure of how balanced a dataset is. Where higher means it's more balanced..

TPR true positive rate.

training set Subset of the dataset where it is used to train the model..

tree Short for tree-based machine learning model..

weight Defines how well a function is fit to a dataset. It is typically minimized in order to find optimal ML model parameters. .

Chapter 1

Introduction

In the past few years AI has taken the world by storm, mainly being powered through machine learning. Some of the applications that have really stolen the spotlight include large-language-models like ChatGPT, art generating models like MidJourney and autonomous vehicles like Tesla. A result of this is that several underappreciated realms in machine learning with numerous benefits to society are often overlooked by the hype of the media due to the applications subjectively not being as 'cool'. This project explores one of those underappreciated domains, being the tabular data domain which is one of the most common forms of data and is incredibly important to a wide range of industries offering numerous benefits to society. It is used in the healthcare industry for disease diagnosis, electronic health records and hospital management. It is used in the financial sector for credit scoring, fraud detection and algorithmic trading. It is used in marketing to make recommender systems and the list of applications goes on[1].

The goal of this project was to compare deep neural networks (DNNs) with **tree**s. Although trees are a relic of time, they are the de-facto model for tabular data, offering superior **runtime** and **score**[2]. Whereas DNNs are by far the best model in all realms of machine learning except tabular data[3]. Various trees and DNNs were explored throughout this project including a very simple model that acts as a baseline. The trees compared in this project were decision trees, random forests, gradient boosting, histogram gradient boosting and the model that is widely regarded in the machine learning (ML) community as the best model for tabular data, XGBoost. Only one DNN was explored in the beginning of the project being the mulilayer perceptron (MLP). The majority of these models were implemented using the popular data science library scikit-learn [4]. XGBoost was implemented using the XGBoost library[5], which is a library mainly used for implementing the various versions of XGBoost. MLP was implemented through scikit-learn for some very basic benchmarks, however for the more time consuming benchmarks I used a far superior library for deep-learning being PyTorch[6]. Which is far more

customizable and allows for GPU support to speed things up. Initially I started this project using the PyCharm IDE however, it quickly became apparent that GPU support primarily for MLP, but for XGBoost as well, was necessary due to the insanely long training times of both models. To access GPU's, Google Colab[7] was used which allowed me to exploit their high-performance cloud GPU's saving on runtime. An additional benefit was that it allowed me to run 12 notebooks at the same time in the background, without holding my computer hostage to always having to be running.

In order to fairly compare trees with DNNs an adequate number of diverse datasets needed to be collected. Numerous diverse datasets are necessary when benchmarking as the more you have the more representative the benchmark becomes of any random unseen dataset and generally becomes more robust to the randomness of the algorithms used to generate the benchmarks. Diversity is necessary as otherwise this benchmark would only be applicable to similar datasets. An additional benefit is that there is less doubt as to whether you cherry picked datasets to forge a result in your favour. This project initially benchmarks the default hyper-parameter models to get a baseline of what putting in zero effort to hyper-parameter tuning would look like. Where the notion of default hyper-parameters is whatever value a hyper-parameter is given as defined by the library application programming interfaces (APIs) when no hyper-parameters are supplied to the model. This gives a good understanding of what model is best when no hyper-parameter tuning is done. Hyper-parameters can drastically affect accuracy and as such it was necessary to compare which models are best after significant hyper-parameter tuning was done.

This project used three hyper-parameter optimization (HPO) techniques being grid search, random search and Bayesian optimization. Where grid search is the least effective technique in efficiently finding optimal hyper-parameters, however it allows you to effectively plot the hyper-parameter search space. Random search is the 2nd most efficient algorithm followed by Bayesian optimization as the most efficient algorithm[8]. Both of these algorithms allow for plotting the effects of increasing amount hyper-parameter tuning with respect to the models accuracy in a concise and fair way.

Finally, this thesis explores a recent, innovative transformer TabPFN [9]. Where not just the type of model is different, but the entire approach to machine learning is different. The authors that made this model claim it achieves higher accuracy in a significantly faster runtime than one of the best models on tabular data being XGBoost. There is a significant caveat to this in that the model is severely restricted in that currently it has only been developed to work on **training set**s of at most 1000 samples,100 features and 10 classes. It also only outperforms trees when the datasets contain no missing values and are purely numerical. To adequately explore the effectiveness of this model compared to the trees, I trimmed my existing datasets

to fit these dataset requirements. Then I reran the default and random search benchmark with these trimmed datasets.

Chapter 2

Background

2.1 Tabular Data

Tabular datasets are typically represented using Excel spreadsheets. Where the rows correspond to the number of samples you have in your dataset. The columns known as features correspond to the attributes of the various samples. Tabular datasets can be composed of 2 different types of features. The first type is purely numerical either of integer or floating point values. For example the price of a vehicle is a feature composed of floating point numbers and the number of seats in the vehicle is composed of integers. The second type of feature is called a categorical feature and can either be a categorical nominal feature or categorical ordinal feature. Ordinal features are features where there is some inherent order in the values such that you can put the values on a number line and nominal features are features where there is no order. For example the size category of the vehicle such as if it is a motorcycle, coupe or a truck is an ordinal feature as a truck is bigger than a coupe and a coupe is bigger than a motorcycle. An example of a nominal feature is the brand of the vehicle as there is no inherent order between brands. Formally speaking, tabular data differs than non tabular data like images or time-series data because the order of the features is irrelevant. For example if you converted an image to a representation where each pixel location is a unique feature and then tried to rearrange the order of the features/pixels, the image would no longer be the same. However if you took the vehicle dataset and swapped the brand feature with the price feature the samples/vehicles would still be from the same brand with the same price.

2.2 Cross-Validation

Cross-validation can be used to get an idea of the score your model achieves on a dataset. Cross-validation works by splitting the dataset into n folds, where each fold approximately contains $1/n * 100\%$ of the dataset samples. In order to get the validation score of a model, you train it using n-1 of the folds and get the score by using the remaining fold not used in the training as the test set. Repeat this process n times, where the training and test folds are always different. The mean score of all the test folds gives us the validation score.

A cross-validation variant known as stratified cross-validation works by splitting the folds in a way where all folds have the same target distribution as the original dataset. The benefit of this is that it reduces the variance in score between folds as if 1 fold was composed of nearly all 1 class then it would not accurately be able to predict the other class, hence likely achieving a significantly worse accuracy.[10]

2.3 Scoring Metric: AUC

AUC, otherwise known as AUROC which stands for 'Area Under the Receiver Operating Characteristic Curve' is the scoring metric I have used throughout the entirety of this project. The receiver operating characteristic curve is made by obtaining the true positive rate (TPR) and false positive rate (FPR) at different thresholds. Where the TPR is defined as the number of true positives divided by the sum of true positives and false negatives. The FPR is defined as the number of false positives divided by the sum of false positives and true negatives. The threshold refers to the threshold used to classify binary classes and can be increased or decreased leading to varying values of the TPR and FPR. When the threshold is increased more samples are negatively predicted and as a result TPR and FPR increase. When the threshold is decreased more samples are positively predicted and as a result the TPR and FPR increase. AUC is the area under this curve and it is the 'probability that the model ranks a random positive [sample] more highly than a random negative [sample]'[11].

I opted to use this over the more beginner friendly accuracy scoring metric, as the pros of AUC far outweigh the cons. The main pro of AUC is that it is robust to class imbalance, whereas accuracy is not. Imagine you have a classifier that always predicts the majority class on a severely imbalanced dataset, where the majority class makes up 99% of the samples. This incredibly dumb classifier will get 99% accuracy on this dataset, as accuracy is just the total number of correct predictions divided by the total number of predictions made. If you were to try and use this classifier in the real world, where the data might be perfectly balanced the accuracy will only be 50%. Hence accuracy is very misleading with regards to imbalanced datasets. AUC

does not have this issue as it only cares about the ranking of predicted probabilities. The major con of AUC is that it is limited to binary classification datasets(with a few exceptions).

2.4 Logistic Regression

Logistic regression is arguably one of the simplest models benchmarked in this project, in that it is the only linear model. It is also neither a tree or a DNN. I have included it to provide a baseline AUC. This model is of the $f = w^T x + b$ form. Where w is a vector of **weight**s corresponding with the dimensions of x and b is the **bias**. You classify samples as class 0 if it is less than some threshold value of f and vice versa.

When fitting ML models there is typically some loss function we want to minimize that reflects how well fit our model is. For logistic regression the loss function is called log loss. If you wanted to get the weights and bias that minimized this, an optimization algorithm is necessary. The most typical one is called stochastic gradient descent (SGD). It approximately works by iterating through every sample of the dataset many times. Where for each sample you calculate the gradients of the log loss with respect to the weights and bias. Updating the weights and bias approximately to itself minus those gradients. Given that log loss is convex this effectively descends the loss function in the opposite direction of those gradients. Covering more 'distance' the further you are from the minimum value, hopefully honing in on the optimum value.

Many models will be explained throughout this background material chapter. This chapter will also cover the hyper-parameter distributions fed to random search and Bayesian optimization for each model. Table D.1 is the hyper-parameter distributions of logistic regression for the HPO algorithms. Refer to the scikit-learn[4] library for details about the default hyper-parameters.

2.5 Deep Learning

2.5.1 MLP

MLP is very similar to logistic regression except that it can have multiple layers with non linear activation functions, hence why it is a 'deep' neural network. Logistic regression is severely limited in that it is only really useful for **linearly separable** datasets. You can overcome this restriction in logistic regression by transforming the data into data that is linearly separable. To do this by hand is incredibly time consuming and requires a lot of knowledge about the dataset. MLP does this feature transformation for you 'automatically', saving immeasurable time and resulting in far better performance.

MLP works by feeding the output of the previous layer as input to the current layer, which eventually passes through the non-linear activation function. The pre-activation output of a layer

is roughly the same as logistic regression having form $f = W^{(l)}h^{(l-1)} + b^{(l)}$. Except that the weights and bias are the weights and bias for that specific layer and the x from logistic regression has been replaced with the outputs from the previous layer. ReLU is a typical activation function that is applied element wise to this pre-activation output which forms the actual output of this layer. ReLU works by setting negative values to zero and leaving positive values untouched.

Training an MLP is somewhat similar to training a logistic regression model in that we are trying to minimize a loss function with respect to the weights and bias except with every layer. SGD can achieve this by calculating the gradients for the loss with respect to the weights and bias of every layer. This involves very tedious math. Thankfully, one of the staples of the library PyTorch is the autograd system which automatically does this for us.

Table D.7 is the hyper-parameter distributions of the MLP for the HPO algorithms and was implemented using the PyTorch library.

2.5.2 TabPFN

TabPFN, where PFN stands for Prior-Data Fitted Network, is a very innovative, DNN developed in 2022.[9]. It is significantly different than the MLP in that it is a pre-trained transformer that requires no hyper-parameter tuning. Instead of training the model with a training set and then being able to use the fitted model to make predictions, TabPFN takes the entire traning set along with a single test sample to make a prediction for that test sample. The two major components that define TabPFN is the prior and posterior predictive distribution. Where the prior is used to generate synthetic datasets that are used to train TabPFN offline only once with. This prior generates synthetic datasets through structural causal models. The posterior predictive distribution of the prior is approximated after a single forward pass of the pre-trained model with the entire training set and a test sample. You can use this to make predictions with.

TabPFN is a hyper-specialized model in that it needs the training set to have less than 1000 samples,100 features and 10 distinct classes. It also does best when the dataset is contains no missing values and is purely numerical. Another downside is that it has rather high inference time.

There is one thing of great interest that the TabPFN paper[9] highlighted that I think is a great benefit. It was that TabPFN does well on datasets where typical models like trees and DNNs do bad and vice versa. This is obviously very beneficial as often you do not care what model you use, so long as you get a high AUC and this means that for more datasets you will be able to get a higher AUC.

2.6 Tree-based models

2.6.1 Decision Tree

Decision trees are the base building blocks that make up the majority of the models within the trees family. Decision trees classify samples by routing them down series of questions(nodes) until it ends up in a leaf. Where a sample going through a node is routed to the left node if its value for a specific feature is less than some threshold value, otherwise its routed to the right node. When a routed sample ends up in a leaf it is classified according to the most probable class in that leaf. Which is determined by the class distribution of the training samples that ended up in that leaf.

You train decision trees in a greedy fashion, in that you start at the root and split it by choosing a feature and threshold value that maximize information gain. Repeat this process for every new node created until one sample is left at which point the node is considered a leaf. Information gain is primarily decided by and inversely proportional to the criterion, which is typically entropy or gini impurity. Entropy is a measure of uniformity and is large when the class distribution is balanced(i.e. uniform) and zero when only one class is present. So in order to maximize information gain you need to minimize entropy, which intuitively makes sense as we are trying to separate classes. Gini impurity is a measure of how likely a random sample from a distribution would be incorrectly classified as the majority class from that distribution. So in a distribution where the majority class makes up the entire distribution, the probability of classifying a random sample, which would be from the majority class, would be 100%, which in turn would make the gini impurity zero. So in order to maximize information gain you need to minimize entropy or gini impurity, which intuitively makes sense as we are trying to separate classes.Entropy and gini impurity more or less measure the same thing and as a result share very similar shapes as they are both parabolic. The choice between either will typically not result in big differences in AUC.

Table D.2 is the hyper-parameter distributions for the decision tree that is fed to the HPO algorithms. It was implemented using scikit-learn.

2.6.2 Random Forest

Random forests are an ensemble of decision trees. They work by training various decision trees in parallel on random variations of the training dataset. This randomness can be implemented in several different ways. The most common method to introduce randomness is called bagging which stands for bootstrap aggregating. Bagging works by allocating each decision tree in the ensemble a random subset of the original dataset to use for training. Another common method is called feature bagging where each node of a decision tree is only allowed to use a random subset

of features from the training set to make the split. The majority classification of all the decision trees is then used to classify a sample. Random forests typically achieve far greater accuracy then decision trees due to the phenomenon known as ‘wisdom of the crowd’. This phenomenon can be seen in action when viewing the TV show ’who wants to be a millionaire’[12] and noting that the ’ask the audience’ lifeline, where every audience member gets to vote on the correct answer, seems to be far more effective than every other lifeline.

Table D.3 is the hyper-parameter distributions for random forest that is fed to the HPO algorithms. It was implemented using scikit-learn.

2.6.3 Gradient Boosted Decision Trees

Gradient boosted decision trees is an ensemble of decision trees and is effectively gradient descent in function space. It works by iteratively adding ’weak’ decision trees(regression) to a ’strong’ ensemble to try and fix the errors of the ensemble. Imagine expressing your classifier(function) as a list of outputs when fed all the training inputs. Take the loss of all these individual points by feeding the loss function the corresponding target values and now you have a list of loss values. For gradient boosting take the gradient with respect to the function outputs(i.e. function space). Now you have a list of gradients of the loss function with respect to the function output for every training point. Somewhat like in gradient descent at the next time step update the function to approximately itself minus the gradient. What I just described is not actually GBDT in fact it is the general version called gradient boosting .GBDT is the specialized version in that at each time step it fits a decision tree to the negative gradients.

Running through the GBDT algorithm would roughly look like having an outer loop that iterates for the amount of decision trees you want in the ensemble. Before you enter the loop initialize the ’strong’ learner(typically zero). At each iteration calculate the gradients and fit a decision tree to the negative gradients. Update the ’strong’ learner to itself minus the fitted decision tree. Then repeat until the loop is done. Another model that I benchmarked throughout this project is histogram gradient boosting. It is very similar to gradient boosting, however it has one major change. It discretizes the input into bins, much like a histogram graph. This results in faster training time, as well as typically higher AUC due to this regularization[13].

Table D.4 and D.5 is the hyper-parameter distributions for the gradient boosting and histogram gradient boosting models respectively, that are fed to the HPO algorithms. Both were implemented using scikit-learn.

2.6.4 XGBoost

XGBoost which stands for Extreme Gradient Boosting was developed in 2014 by two University of Washington students[14]. It has built up quite the reputation over the years by being the de-facto model for winning Kaggle competitions. It is far more efficient and scalable than the standard gradient boosting algorithms due to several optimizations. It includes L1 and L2 regularization, it allows for distributed computing allowing for scalability and it handles sparse or missing data.

XGBoost differs to other gradient boosting algorithms in that it uses a mix of Hessian's and gradients to update the leaf values of the trees, Where the Hessian in this case is the second derivative of the loss function.

Table D.6 is the hyper-parameter distributions for XGBoost that is fed to the HPO algorithms. It was implemented using the XGBoost library. Refer to said library for details aboyt the default hyper-parameters.

2.7 Model Interpretability

Intepretability is an incredibly important when choosing what model to use. It refers to how easy it is to understand how an ML model makes its decisions.

Logistic regression is one of the most interpretable models given that each weight is proportional to how much a feature affects its decisions. [15] DNNs are notoriously uninterpretable black boxes and MLP and TabPFN are no exception to this due to their sea of interconnecting parameters. They the most uninterpretable models in this benchmark.

Tree-based models generally are more interpretable than DNNs, but of course depends on which tree you are discussing. Decision tree is the most interpretable tree-based model and just as interpretable as logisitc regression, but in a different way. To understand how decision trees make decisions, just look at how it thresholds samples which is quite easy to understand.[15] Random forest is somewhat interpretable as there are vast amount of trees to look through. Gradient boosting,histogram gradient boosting and XGBoost is less interpretable than random forest as the serial connection of the trees makes it very tough to follow.

2.8 HPO Algorithms

Hyper-parameters directly influence how a ML model works. They are not learnt during training as they are supplied to the model before training by the user, hence the prefix hyper. HPO algorithms are used to efficiently search for optimal hyper-parameter configurations of the models. Throughout this project I explored several HPO techniques, however I only used the most

appropriate ones for a comparative project. The HPO techniques I used in this project are grid search, random search and Bayesian optimization.

2.8.1 Grid Search

Grid search works by supplying all the hyper-parameters for a model that you want to explore along with a list of possible values for said hyper-parameters. It works by exhaustively cross-validating the supplied model with all possible combinations of the supplied hyper-parameters. For example, if you supplied a decision tree classifier as a model with max depth and max features as hyper-parameters with possible values of [1,2] and [4,5] as possible hyper-parameter values respectively. The grid search algorithm would fit a decision tree classifier to every [(max depth,max features),(1,4),(1,5),(2,4),(2,5)] hyper-parameter/tuple configuration and record various results for each configuration. Such as time to fit the model and most importantly the mean score of the folds.

2.8.2 Random Search

Random search works by supplying all the hyper-parameters for a model that you want to explore along with a distribution for said hyper-parameters. It works by sampling hyper-parameter values from the provided distributions and fitting a model to each sampled hyper-parameter configuration. For example, if you supplied MLP as a model with alpha and learning rate as hyper-parameters with distributions of a log-normal distribution with a mean m and variation sigma and uniform distribution with endpoints 0 and 1 as possible hyper-parameter distributions respectively. The random search algorithm would fit a MLP to n hyper-parameter configurations sampled from the distributions where n is how many configurations you want to sample and record various results for each configuration such as time to fit the model and most importantly the score for each fold along with the computed mean score of the folds.

2.8.3 Bayesian optimization

Bayesian optimization is a supposedly more efficient HPO algorithm compared to random search as it actually tries to make decisions to efficiently search the hyper-parameter space[8]. Bayesian optimization works by using three functions being the objective function,surrogate function and the acquisition function. Where the objective function is the true loss function with respect to the hyper-parameter search space. Which is obviously unknown(black box) as otherwise we would immediately know what the optimal hyper-parameter configuration is. The surrogate function is a function that has been fit to the objective function as every iteration of Bayesian optimization we will know an additional sample of the true objective function. It is possible to

use different kinds of models for the surrogate function, however the most common surrogate function is a Gaussian process and is the one I used in my implementation, given that I used the library scikit-optimize[16] to implement Bayesian optimization.

A Gaussian process is very different than all the other ML models I just mentioned and I will briefly talk about how it works in Bayesian optimization. It is a set of random variables over all hyper-parameter configurations. Where the output is the AUC and it is a random variable and all of these random variables have multivariate Gaussian distributions. The mean and kernel function is our priori assumptions about what we think the true function we are trying to fit looks like. The mean function represents the expected value for all the random variables and the kernel function defines the covariance between these random variables. The kernel is far more interesting and a typical kernel is when points are close together they are highly correlated and vice versa. For a Gaussian process to actually be useful we need a way of fitting it to data. So when in Bayesian optimization we are given the true objective values with corresponding hyper-parameter configurations we can condition the Gaussian process with these data points,essentially fitting it to these points. This allows us to make predictions as when we marginalize this Gaussian process the mean will be used for predictions and the variance tells us the uncertainty in the prediction.

The acquisition function is a function that returns the hyper-parameter configuration that maximizes itself. This is the 'decision' part of Bayesian optimization, where this function balances exploration of the true objective function with exploitation. Where for exploration it would return hyper-parameter configurations where the surrogate function has high uncertainty about the objective function. For exploitation this would be where expected local high performance of the optimization function is. Obviously a balance of exploration with exploitation is necessary as if you solely went for exploration you would sample highly uncertain locations, however it is likely that none will have good performance. If you went solely with exploitation you will get the best performance locally, however it is very likely you will miss the global optimum.

Bayesian optimization works by combining these three functions in a loop. The number of times you want the surrogate function to be sampled is supplied to Bayesian optimization and controls how many times you loop. At every iteration the true objective function has been calculated for the hyper-parameter configuration that maximized the acquisition function from the previous iteration. So at n iterations you will know the true objective function at n points(hyper-parameter configurations). Then fit the surrogate function(Gaussian process) to these points and find the hyper-parameter configuration that maximizes the acquisition function. Repeat this loop for however many hyper-parameter configurations you want to try[17].

2.9 Related Work

The paper most inline with this project and inspired how I approached this project is called 'Why do tree-based models still outperform deep learning on tabular data'[2]. This paper did several benchmarks using some of the same models that I used. They used random search on regression and classification datasets of medium($<$10000 samples) to large($<$50000 samples) sample sizes. Their results showed that trees were vastly superior to DNNs on regression and classification datasets irrespective of the size of the datasets. The reasons they found that explained the superiority of trees were that DNNs are biased to smooth solutions, that uninformative features affect DNNs more than trees and tabular data is rotationally invariant whereas the training process for some DNNs are not.

The paper titled 'Well-tuned Simple Nets Excel on Tabular Datasets' is inline with the last main task of my mission statement[3]. Unfortunately, I was unable do this task as the code accompanying this paper did not work. However this paper still provides great insight into the capability of DNNs on tabular data when tuned properly. It shows that an MLP that has been regularized using an algorithm to search through a 'regularization cocktail' outperforms XGBoost. Where the regularization cocktail is a hyper-parameter search space of 13 different regularization techniques.

One of the most valuable resources I encountered when doing this project was a blog that keeps up to date with the current progress of deep learning for tabular data. It posts some of the most important papers along with brief summaries of the papers. It is maintained by world renowned machine learning researcher Sebastian Raschka. This blog is well worth a read.[18].

Chapter 3

Dataset Information

3.1 Dataset Collection

44 datasets were collected for this project from various sources. The information for all these datasets is presented in table B.1. 15 datasets were collected from the free to use website Kaggle[19], 22 datasets were collected from a website the University of California, Irvine(UCI) maintains[20], 6 datasets were collected from Murat Koklu's website[21] that I found through Kaggle who is a reputable professor at Selcuk University. 1 dataset was from my Principles and Design of IoT Systems(PDIOT) course.

Kaggle is a data science website founded by Anthony Goldbloom in 2010 where data science enthusiasts can post datasets and models they have created. People can also partake in discussions about other people's datasets as well as post code and the results they have gotten for any dataset. Kaggle is very famous for hosting data science competitions with lucrative rewards ranging up to millions of dollars. These competitions revolve around all the sub-fields within data science. Some currently trending competitions at the time of writing this are the AI Mathematical Olympiad[22] with a total prize fund of 10 million dollars where the competitors are tasked with creating a model capable of solving complex math problems. Another trending competition that is more inline with this project and I might even be able to attempt with the knowledge I have picked up over the course of this project is the credit risk competition[23]. It has a prize purse of $105,000 where competitors are tasked with using ML techniques to predict which people will likely default on their loans.

The website maintained by UCI was created in 1987 by UCI PhD student David Aha and is a lot more limited than Kaggle in that it is strictly a dataset repository of much smaller size, however it overcomes a significant challenge in collecting datasets. Where the challenge was in making sure that they actually came for genuine sources and were not artificially created. UCI

is far more moderated than Kaggle as you need to submit a dataset where the UCI team will either allow it to be publicly displayed on the repository or deny the dataset. Whereas with Kaggle anyone with an account can submit a dataset with no external moderation whatsoever. Artificial datasets obviously compromise the integrity of this benchmark as than this benchmark is not representative of datasets you would naturally encounter.

I found Murat Koklu's website through some of his datasets that he posted to Kaggle. Of which I was very impressed with the quality of his datasets and the accompanying papers, hence why I included so many of his datasets. The majority of his datasets have been generated by taking photos of various kinds of produce and then extracting various different tabular features through image processing techniques.

The dataset I got from my PDIOT course last semester was generated as result of the 1st coursework of that course. Where for that coursework all students were tasked with recording 30 seconds of data for 22 different activities. We recorded the data through a wearable accelerometer sensor called a Respeck device[24] that was developed by University of Edinburgh Professor D K Arvind. I turned this time series data into tabular data by calculating very typical statistical features like min,max,skew,quantiles... etc for the x,y and z channels of the accelerometer.

3.2 Dataset Preprocessing

In transforming my datasets into appropriate datasets for machine learning I had to appropriately deal with the null values in some of the features. There are typically two approaches used to deal with null values. The first is that you can either delete features with any null values or you can delete samples with any null values. Deleting features vs samples typically depends on which will result in less destruction of information. The second method of handling null values is to impute values. This involves replacing null values with the mean of the features for numerical features or replacing with the mode of the feature for categorical features. The majority of my datasets contained no null values, however for the ones that did I opted to delete samples or features. I did whichever would result in the least amount of information deleted. I did this even if imputing would result in higher AUC values as I only care about the relativeness of the models AUC and it is somewhat artificial when imputing values in that the samples with imputed values never existed in the first place.

Standardizing features is an absolute necessity for the MLP and logistic regression model and is not necessary for any of the trees. So I standardized all numerical features and non-binary ordinal features. I did this using the scikit-learn provided StandardScaler transformer[25]. Which brings all features to the same relative scale by transforming them to have zero mean and unit variance.

Machine learning models can not take strings(native categorical features) as values so you need to appropriately numerically encode them. For ordinal features I manually encoded the value that was the lowest in order as 0 and the value greatest in order as n where there are n distinct values. For nominal values I transformed them to one-hot code representation using scikit-learns OneHotEncoder[26] transformer. Which transforms a feature into n features where n is the number of distinct values in that feature and one hot encoding these features appropriately. To reduce the amount of one hotted features while still retaining all the information of the one hotted features, I dropped the last feature. This still retains all the information as to represent that last feature all other features are set to 0. The main benefit of doing this was to reduce training time as features directly influence training time.

I initially collected 44 datasets where some where suitable for regression and some where suitable for multi-class classification. Part way through my project I decided to use only binary classification datasets so that I could use a very robust scoring metric AUC.It is not necessary to benchmark regression or multi-class classification as the results from binary-classification will be representative enough of regression and multi-class-classification given how similar all three paradigms are. In order to not waste the 44 datasets that I already collected, I decided to bucketize them into binary classification datasets.

When bucketizing I prioritized bucketizing in sensible ways. All of the information and statistics of the various datasets I collected is presented in figure B.1,including the mean statistics of all these datasets. For example, the CSVAbalone dataset where the target is the number of rings on the abalone, you could bucketize it such that if the number of rings is less than 5 or above 25 it's in class 0 otherwise it's in class 1. Clearly the more sensible way of bucketizing this dataset is to pick a single threshold value of 15 and if it's above, it's class 1 else it's in class 0. This indirectly results in higher AUC values achieved by the models as the features will typically be more in common for closer target values. This also applies to categorical classification datasets like the PDIOT dataset, where there were 22 classes, 10 of which were various physical stationary activities and the other 12 classes where non-stationary activities. I bucketized this dataset into stationary and non-stationary activities. It is important to sensibly bucketize datasets because you are typically trying to predict a sensible thing. So therefor these datasets will be more representative of the typical dataset you will encounter. My second priority was achieving a balanced dataset(high **Shannon Entropy**). Even though AUC is robust to imbalanced datasets there is literally no downside in trying to make them more balanced.

Due to the budgeting constraints of this project, to save time I chose to trim any dataset over 10000 samples to 10000 samples. I tried to trim in a way that would lead to the datasets being more balanced target wise. Overall the datasets had an small to medium amount of samples with a mean sample size of 3472.2 samples. The datasets had an average amount of features

with a mean number of features of 26.5. The majority of datasets were perfectly balanced with a Shannon Entropy of ~ 1.0 except for the lowest dataset being zCompanyBankrupty with a Shannon entropy of 0.206 which brought the mean Shannon entropy to 0.925, which is still extraordinarily high.

3.2.1 TabPFN Dataset Preprocessing

TabPFN is limited to datasets of less than 1000 samples for the training set, 100 features and 10 classes[9]. So I went through all of the 44 datasets from the previous section and trimmed all the datasets not meeting that criteria to 1500 samples and 100 features. I trimmed them to 1500 samples and not 1000 samples as 1000 were for the training set and 500 for the test set given that I always used 3 fold stratified cross-validation throughout this project. It was not necessary to check if they were over 10 classes as they are all binary-classification. TabPFN excels when all the features are purely numerical and although this is not a requirement it does not make much sense to not account for this. As there is not point in benchmarking if you are not allowing every model to be the best that they can be. So I deleted every nominal feature. I also deleted every ordinal feature not having more than 20 distinct values. The line can get pretty blurred in deciding whether a feature is ordinal or numerical, however the more distinct values an ordinal feature has the more alike a purely numerical feature it is. So using 20 distinct values as the threshold is a good rule of thumb. Sometimes this would result in outright deleting a dataset if there were no purely numerical features or the ratio of numerical to categorical features was too low. As I figured deleting the majority of features in low ratio datasets would achieve terrible AUC and as a result I would delete them. This resulted in 29 trimmed datasets remaining from the original 44 datasets. Unlike the MLP and logistic regression you do not need to manually standardize the datasets as it automatically sets the features to zero mean and unit variance[9].

Overall the trimmed datasets had a very small amount of samples with a mean sample size of 1023.3 samples. The datasets had a small to average amount of features with a mean number of 17.59 purely numerical features. The majority of datasets were perfectly balanced with a Shannon Entropy of ~ 1.0 except for the lowest dataset being HCV with a Shannon entropy of 0.491. This brought the mean Shannon Entropy to 0.955 which is still extraordinarily high.

Chapter 4

Benchmark Results

All of the results in this chapter are with respect to the 44 datasets I have collected.

4.1 Default Hyper-parameter Models

The tree-based models benchmarked in this section were decision tree, random forest, gradient boosting, histogram gradient boosting and XGBoost. All of which, except XGBoost, were implemented using the scikit-learn library and XGBoost was implemented using the XGBoost library. One DNN was benchmarked being the MLP and one baseline model was benchmarked being logistic regression, of which both were implemented using the scikit-learn library. Refer to the code snippet in listing A.1 for details of the implementation.

Before I did the random search benchmark. I felt it was necessary to get a baseline AUC of what the default hyper-parameter configurations for the models might be. These hyper-parameter values are set to default when no hyper-parameters are supplied to the model. As I implemented all the models using the scikit-learn and XGBoost libraries, all of the default hyper-parameter values can be found in their respective APIs. Throughout this project I used various libraries to implement MLP and for the default hyper-parameters benchmark I used the scikit-learn library as this library has more of a concrete notion of default hyper-parameters. Whereas the library I used in random search and Bayesian optimization does not have as concrete a notion of default hyper-parameters. In order to implement the default hyper-parameters benchmark I used 3-fold stratified cross validation for all the models, using AUC as the scoring metric to get the cross-validation AUC for all 44 individual datasets. I calculated the mean AUC and rank across the 44 datasets including the +- standard deviation bands which is presented in table 4.1.

Default Model	Mean AUC	Mean Rank
Logistic Regression	0.862 +-0.136	4.04+-2.11
Decision Tree	0.760 +-0.144	4.09+-2.06
Random Forest	0.873 +-0.143	4.63+-1.65
Gradient Boosting	0.875 +-0.141	2.31+-1.18
HistGradient Boosting	0.869 +-0.150	4.45+-1.78
XGBoost	0.872 +-0.143	4.56+-1.94
MLP	0.876 +-0.126	3.88+-2.09

Table 4.1: Mean AUC and rank of 44 Datasets for default hyper-parameter models(with +-standard deviation)

All of the tree-based models(except decision tree) and the DNN MLP have the same mean AUC on the 44 datasets of ~ 0.87 AUC. This is a very respectable AUC value and suggests that the majority of these datasets are appropriate for machine learning tasks. Logistic regression is slightly worse then the best trees, although this difference is negligible. Decision tree is substantially worse at ~ 0.1 AUC lower than the other models at ~ 0.76 AUC. This is still a decent AUC score and if you were forced to use a decision tree these datasets would still be appropriate for machine learning. It is not unexpected that the decision tree is by far the worse model as they notoriously over-fit to data [27]. Logistic regression being the 2nd worse model is also not unexpected as it is the one of the simplest model's being the only model benchmarked that is linear. Therefor from AUC alone DNNs achieve equivalent performance to trees.

The AUC standard deviations of all the models are also approximately the same at +-0.14 AUC. This is a somewhat high standard deviation considering that for AUC values you will typically only have values between 0.5 AUC and 1 AUC[28]. Meaning that $\sim 68.27\%$(1st standard deviation) of the 44 dataset AUC values for all the models lie within 54% of the typical 0.5 to 1.0 AUC range and all of the models except decision tree have $\sim 68.27\%$ of the values lying between ~ 1.01 AUC and ~ 0.73 AUC. The only real conclusion you can draw from these standard deviations is not so much related to the relativeness of the models, but rather the datasets themselves or absolute AUC of the models. Where some models for some datasets are very suitable for machine learning predictive tasks and some are not. If some of the models had really small standard deviations this would tell you that you could expect the model to get the same AUC irrespective of the dataset.

When benchmarking models the mean rank is far more important than the mean AUC as it only cares about the relativeness of the models. The ranks are quite unusual in that the supposed best tree being XGBoost has a rank of 4.56 and the supposed worse tree being decision tree has a better rank of 4.09. What is even more strange is that XGBoost has an AUC that

is 0.112 higher than the AUC achieved by the decision tree. This suggests that XGBoost does substantially better than decision tree on a couple datasets, whereas decision tree marginally beats XGBoost on many datasets. There are several other strange anomalies with the rank, however lets focus on comparing trees with DNNs.

The model with the highest rank is the tree, gradient boosting with a rank of 2.31. This is substantially better then the DNN MLP with a rank of 3.88. Given this substantial difference of 1.57 rank between the best tree and best DNN it is safe to say that trees somewhat outperform DNNs when using default hyper-parameter models. The reason why I say trees are better than DNNs even though they achieve the same AUC, is that there are likely to be outliers that are causing the AUC values to be the same and the rank is robust to outliers.

The rank standard deviations contain very valuable information about the models. In that if they have high standard deviations this suggests that the models are very sensitive to the datasets. In that sometimes they are the best model and sometimes they are the worst. Whereas low standard deviations imply that the model is invariant to the datasets and always achieves the mean rank. This information is somewhat irrelevant in deciding whether trees are better than DNNs. However given that this project is not just about comparing the performance of trees with DNNs but generally comparing them. This means the standard deviations are still of interest. Generally, I would rather a model with a low rank standard deviation and a low AUC standard deviation as there is less risk attached and will reliably perform how you expect. This reasoning is similar to why some stock traders avoid volatile stocks[29].

It is also interesting to note that the standard deviations for the rank are already somewhat baked into the mean ranks. In that if several models have the same mean rank their standard deviations will likely be high and vice versa.

The rank standard deviations vary far more than the AUC standard deviations as the tree gradient boosting has the lowest standard deviation of 1.18 and the highest is for the DNN MLP and logistic regression which is nearly twice as high. The average standard deviation of about 1.5 for these models is somewhat high considering that the valid rank range is 1 to 7. So about 68.27% of rank values lie within 50% of this range. These standard deviations suggests the rank varies between datasets significantly for the DNN MLP and less for the trees.

To sum up this section trees are better than DNNs when using default hyper-parameter models. As they achieve substantially better rank than DNNs. Even though they achieve the same mean AUC, the mean rank is far more important as it is robust to outliers.

4.2 Random Search

Random search is a good way of visualizing the effects of increasing the HPO budget with respect to some metric. As it keeps the amount of effort to hyper-parameter tuning each model the same. There are 3 types of plots that I have generated from my random search benchmark that reveals different types of information about the effects of an increasing HPO budget. For every HPO plot produced in this thesis I did the same post processing step to the HPO algorithm's results. Where the value for a model at a given iteration on the x-axis is the maximum value of every iteration before it, including itself. Otherwise the plots would just look like static noise and would not reveal any meaningful information. These plots say 'this y-value is the best value achieved by this HPO algorithm after this many iterations on the x-axis'. Therefor you are able to directly compare models as each iteration signifies the same amount of hyper-parameter tuning done to each model. This also means all the model HPO curves never decrease.

The first type of plot I have produced is presented in figure C.1 and is the HPO curve of each model with respect to AUC for 44 individual datasets, resulting in 44 subplots. Where the number of iterations of random search is on the x-axis and the AUC achieved by the models is on the y-axis. The second type of plot I produced is the mean AUC plot of all those 44 individual AUC plots, which is presented in figure 4.1. The last type of plot I generated is the mean rank with respect to the number of iterations of random search, which is presented in figure 4.2. Where, similarly to the mean AUC I generated the individual ranks of all the models for all 44 individual datasets for all 300 iterations. Then I took the mean of all 44 ranks for all 300 iterations. Obviously, there is no point showing the individual dataset rank plots/calculations as the information is already in the individual AUC plots.

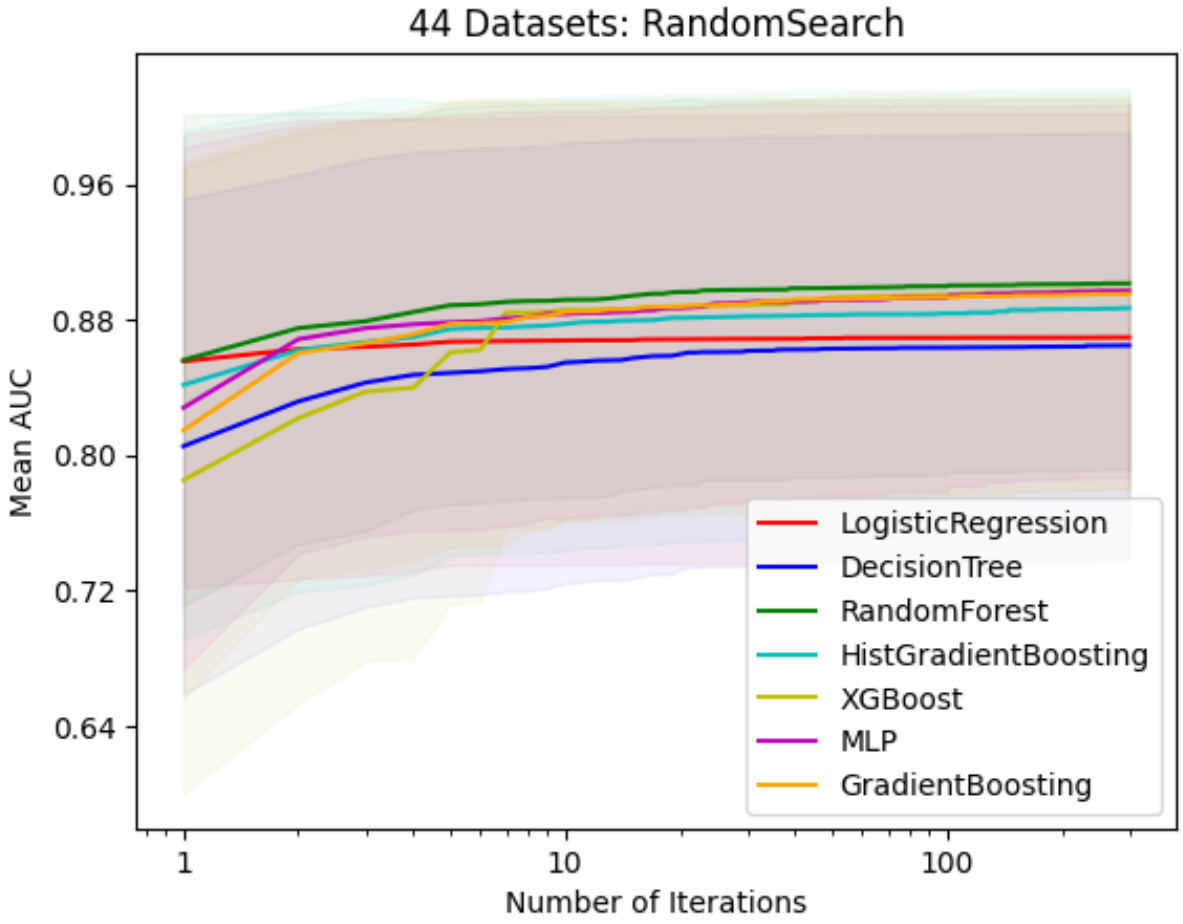

Figure 4.1: Mean AUC of random search results with respect to the number of iterations for 44 datasets(with shaded +- standard deviation bands).

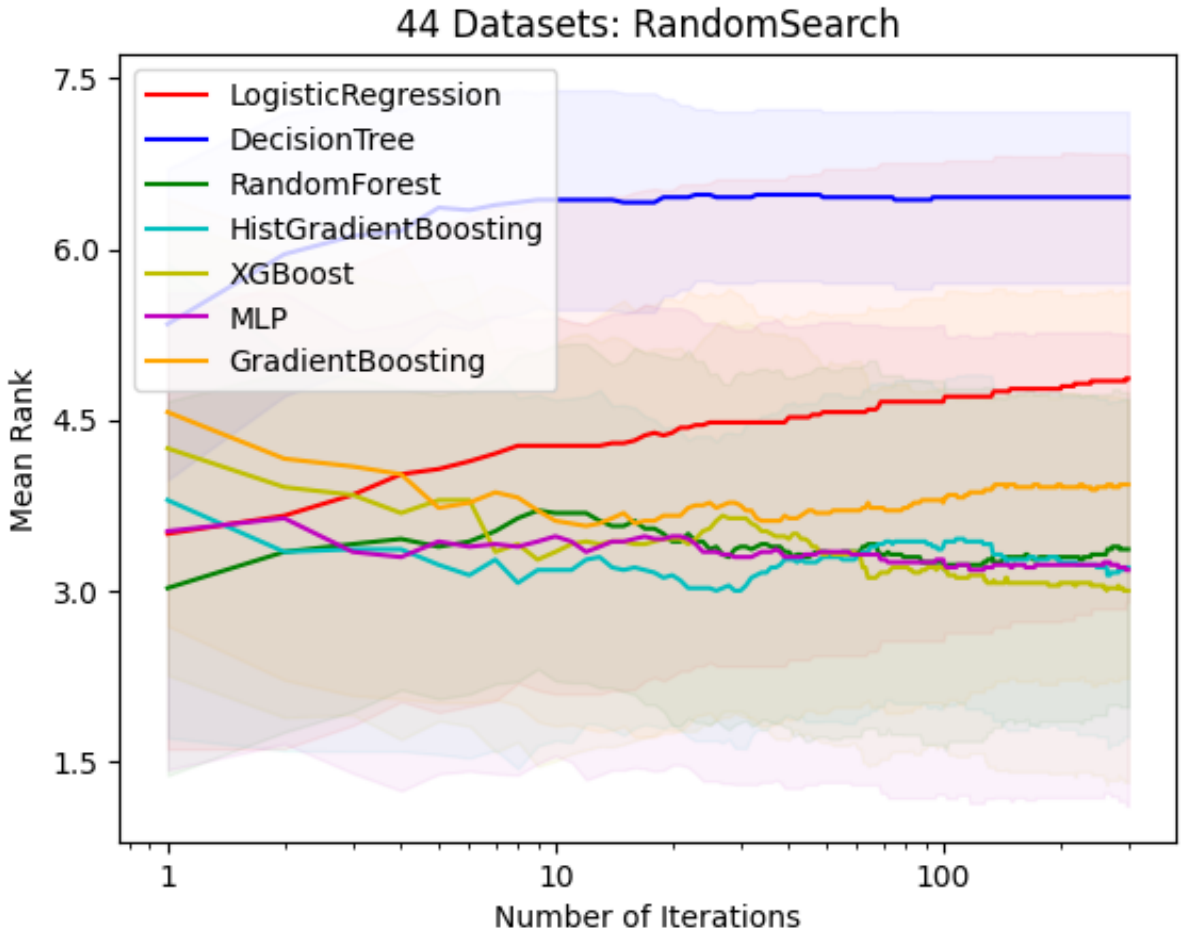

Figure 4.2: Mean rank of random search results with respect to the number of iterations for 44 datasets(with shaded +- standard deviation bands).

The individual plots are particularly useful for outlier/anomaly detection in not just the models, but the datasets as well. There are quite a substantial number of 'easy' datasets where the simple linear baseline model logistic regression does as well as the other complex models. The Accelerometer Cooler Fan dataset appears to not be very lineraly seperable as logistic regression has an AUC of ~ 0.48 for all the iterations, while all the other models get above 0.9 AUC for the majority of iterations. Hardly any of the datasets have all the models getting either 1.0 AUC or 0.5 AUC throughout the majority of iterations. Which is good as hyper-parameter tuning is not useful in these cases. These plots contain quite a lot of information, however because of this, it is tough to get a concise understanding of whether trees are better than DNNs. So the other types of plots are far better for that.

The mean AUC plot presented in figure 4.1 is very useful for a concise way of understanding

how an increase to the number of iterations affects the models performance. It shows the AUC you can expect to get for the models after n iterations with respect to the 44 datasets. The standard deviation bands is not as useful for selecting the best models, as they tell you more about the datasets. The problem with the mean AUC plot is that although there is somewhat relativity of the models, it is far more about the absolute values achieved by the models. This means that the mean AUC plot is not very robust to outliers and given the random nature of random search, there are bound to be outliers.

This is where the mean rank plot presented in figure 4.2 is useful as it only cares about the relativity of the models. At every iteration you can get an idea of what the mean rank of the models are. With the combination of these 3 plots, you can come to robust conclusions about the effectiveness of each model and whether trees are in fact better than DNNs.

In creating all the HPO benchmarks throughout this thesis I had a couple concerns. The first was how to ensure that your are putting in an equal amount of effort in hyper-parameter tuning to each model. As if you put in considerable effort in finding the optimal hyper-parameter distributions for one model while using sub-par distributions for another this would obviously bias the benchmark. It is naturally very tough to define what putting in effort is, however a good way to ensure fairness is by looking at a reputable paper and use whatever hyper-parameters and distributions they use. Which is what I have done.

My second concern was how to ensure I was not wasting any of my limited computing resources. As there is no point in allocating a massive amount of iterations for the HPO algorithms when you are only tuning one hyper-parameter with a small distribution. As the HPO algorithms would plateau after the first couple iterations effectively wasting all the other iterations. On the other hand it is not wise to provide an excessive amount of hyper-parameters as you would be leaving a lot up to randomness, in turn likely generating several outliers. Given that this is a comparative project minimizing randomness and outliers is imperative. An easy way to combat this is like before by using a reputable papers distributions along with the budget of iterations they use. While making sure their results show that their budget was appropriate for the breadth of their hyper-parameters.

To ensure fairness for each model I adapted my selection of hyper-parameters and distributions from a reputable paper that also benchmarks various models on tabular data.[2]. This paper uses random search with a budget of 400 iterations per model. I used a slightly less amount of iterations per model of 300 to save time. The budget of 400 iterations this paper used seems to be appropriate as their HPO plots show the models plateauing towards the last 50 iterations. To account for my smaller budget and to ensure my hyper-parameters are appropriate for my budget of 300 iterations. I excluded any hyper-parameters from the paper that had a relatively low rf importance score or very negative lin coef score as defined on page 31 of the

paper. Where rf importance "gives us a score which [represents] how much [a] hyper-parameter should be tuned" and lin coef measures "if [a] hyper-parameter has a positive or negative impact on the performance"[2]. Clearly, unimportant hyper-parameters or hyper-parameters that negatively impact performance are a waste of budget. I also trimmed some of the hyper-parameter distributions that would directly increase runtime like the maximum number of decision trees in a random forest as I was trying to cut down on time. Given that I slightly adapted the distributions from the paper this does not inherently mean I have unfairly given more effort to any one model as I have adapted all the models using the same approach. For the hyper-parameters and corresponding distributions I used for random search refer to the tables in appendix D.

In determining whether trees are better than DNNs I will use a combined approach. By using the various HPO plots I have produced as well as table 4.2, which takes a slice of the mean rank and mean AUC plot at the 300th iteration. Which I have included for clarity purposes and it allows you to easily compare with the default benchmark. I think the results at the 300th(final) iteration are more important than what happens before the 300th iteration. As it is very likely we are comparing the optimal hyper-parameter configurations for each model of every dataset. Of which the indivdual plots corroborate this by seemingly all plateauing before the 300th iteration. I will use the same approach in the Bayesian Optimization and TabPFN benchmark as well.

Model	**Mean AUC**	**Mean Rank**
Logistic Regression	0.869+-0.132	4.86+-1.96
Decision Tree	0.864+-0.125	6.45+-0.75
Random Forest	0.901+-0.109	3.36+-1.38
Histogram Gradient Boosting	0.886+-0.129	3.20+-1.48
Gradient Boosting	0.895+-0.115	3.93+-1.69
XGBoost	0.896+-0.117	3.00+-1.67
MLP	0.897+-0.109	3.18+-2.07

Table 4.2: Mean AUC and rank of 44 Datasets (with +-standard deviation) at the 300th iteration of random search.

From looking at the mean AUC plot, it is clear that no matter the iteration, trees are marginally better than the DNN MLP as random forest always has the highest AUC. This superiority margin stays fairly constant throughout all iterations. All of the models seem to be plateauing at the same rate. Except logistic regression which severely plateaued early on which makes sense given I only allowed it 1 hyper-parameter to tune. All the standard deviations of the models appear to be somewhat the same and constant no matter the iteration.

From looking at the mean rank plot, it is clear that no matter the iteration, trees are marginally better than the DNN MLP as at least one tree always has the highest rank. The MLP rank is fairly invariant to hyper-parameter tuning while the majority of trees seem to improve with increased tuning. All the standard deviations of the models appear to be somewhat the same and constant no matter the iteration.

All the models except logistic regression, decision tree and histogram gradient boosting achieve an AUC of ~ 0.9. Logistic regression and decision tree achieve significantly worse AUC of ~ 0.865 which is expected as decision trees notoriously overfit[27] and logistic regression is the only linear model, meaning it is unable to make to complex decision boundaries. Histogram gradient boosting is slightly better than logistic regression and decision tree with an AUC of 0.886. The highest AUC achieving tree-based models on this benchmark being XGBoost,gradient boosting and random forest all achieve the same AUC as the DNN, MLP being ~ 0.9 AUC. All of the models have standard deviations of around ~ 0.11 AUC and do not particularly help decipher whether trees are better than DNNs. Therefor from the AUC alone the DNN MLP is just as good as the best tree-based models.

From the rank it is clear that on this benchmark trees are marginally better than the DNNs. As the highest ranked model being the tree-based model XGBoost achieves a rank of 3.0 compared the best DNN, MLP with a rank of 3.18. Previously when I was discussing AUC I said the difference in AUC between many models was negligible, however this difference in rank is somewhat significant and not negligible. The standard deviations of the rank are fairly large at around 1.5 suggesting they are not invariant to the datasets. The decision tree having the lowest standard deviation of 0.75 is not surprising either considering that it is by far the worst model so it always does the the worst irrespective of the dataset.

With the combined information provided by the mean AUC and mean rank at the final iteration it is clear that trees are marginally better than DNNs. Even though the highest AUC achieving tree achieved the same AUC as the best DNN, the highest rank achieving tree achieved a significant increase in rank compared to the highest achieving DNN. The rank is far more informative for which models are best as there are bound to be outliers given that random search is a random process and the AUC plot is not robust to outliers whereas the rank plot is.

Now there are the questions of whether hyper-parameter tuning these models using random search was even worth it and if hyper-parameter tuning widens or shrinks the superiority gap of trees with DNNs. Using default models is $\sim$ 300 times faster than random searching and the models that achieved $\sim$ $0.9AUC$ using random search achieved $\sim$ 0.875 AUC using their default models. This is a pretty significant increase and I would say hyper-parameter tuning is worth it, but once again it is up to whoever is reading this if the time investment is worth it for their application needs. To answer the second question, for the default hyper-parameters the

difference in rank between the best tree being gradient boosting and the best DNN being MLP was 1.57 in favour of the tree. For the tuned models at the 300th iteration the difference in rank between the best tree being XGBoost and the best DNN being MLP was 0.18 in favour of the tree. So therefor hyper-parameter tuning shrunk the superiority gap between trees and DNNs. The AUC is irrelevant given that the best tree has the same AUC as the MLP regardless of if they were hyper-parameter tuned or not.

4.3 Bayesian Optimization

The reasoning for doing a Bayesian optimization benchmark is that like random search it is also widely used in industry. So getting a benchmark of which models do best when the HPO algorithm is Bayesian optimization is quite valuable. It also allowed me to explore a different budget of iterations per model along with different hyper-parameters and distributions.

When implementing Bayesian optimization I considered using the same hyper-parameter distributions with the same number of iterations allocated to each model as with random search. This would allow me to directly compare the effectiveness of Bayesian optimization to random search and find out which HPO algorithm is most effective. However given that this project is about comparing trees with DNNs and not HPO algorithms I decided to use differing hyper-parameter distributions. The benefits of using differing hyper-parameters to random search is that the AUC/rank of models will likely change based on the breadth of hyper-parameters and distributions. So getting a benchmark for a differing range of hyper-parameters is very valuable. I used the same number of iterations per model as random search, being 300 iterations. Given that Bayesian optimization is supposedly more effective than random search when using the same number of iterations per model[8]. This allowed me to increase the breadth of hyper-parameters and distributions. If I did not the results would likely plateau far sooner than in random search given the increased effectiveness of Bayesian optimization.

The hyper-parameters and distributions I used for the models were adapted from the paper titled 'Why do tree-based models still outperform deep learning on tabular data'[2]. If you recall from the previous random search section I also used hyper-parameters from this paper. However, for random search I narrowed the ranges of some of the time intensive hyper-parameters and outright excluded hyper-parameters that were unimportant or negatively affected performance. For Bayesian optimization I did not do this and I used identical hyper-parameters and distributions to the ones in the paper(with a couple exceptions). However I did not just limit myself to only these hyper-parameter as I added a few more for each model and the reasoning for why will soon become apparent.

This paper and a significant number of other papers doing benchmarks on tabular data all

adapted(shrunk) their hyper-parameters and distributions from the paper titled 'Tabular Data: Deep Learning is Not All You Need'[1]. This paper used the HPO tuning library HyperOpt[30] to generate their benchmarks, where HyperOpt uses Bayesian optimization. They used a budget of 650 iterations per model to tune the hyper-parameters. The loss values of their models plateaued severely after about 100 iterations. Given that HyperOpt gives effectively similar performance to the HPO tuning library scikit-optimize[16] that I used to implement Bayesian optimization. This allowed me to increase the the breadth of hyper-parameters and distributions from the 'Why do tree-based models still outperform deep learning on tabular data'[2] paper so that the HPO plots plateaued towards the 300th iteration as opposed to the 100th iteration. The additional hyper-parameters that I added to the original papers selection of hyper-parameters were obtained from various other papers that seemed to be the most important hyper-parameters. This was not ideal as I would have rather used a papers exact hyper-parameters, however given that after looking through dozens of papers with none that matched my budget with appropriate hyper-parameters, I did not have a choice. I have still maintained somewhat equal effort in hyper-parameter tuning to each models as I did the same approach with each model.

With the individual plots presented in figure C.2 there are a few outliers/anomalies such as in the Date2nd dataset which has models either at 1.0 AUC or 0.5 AUC. There appears to be a tipping point to where if it is met, the AUC snowballs into 1.0 AUC. There are also quite a few datasets like the parkinsons dataset where in the first couple of iterations some models achieve an AUC between 0.4 and 0.5. Sometimes this indicates an error however given that there are bound to be some very sub-optimal hyper-parameter configurations this is not unexpected. Like with the random search results there are a similar number of 'easy' datasets.

Like with the random search results I have also produced the mean AUC plot which is presented in figure 4.3 and the mean rank plot which is presented in figure 4.4. Along with the corresponding mean AUC/rank table at the 300th iteration which is presented in table 4.3.

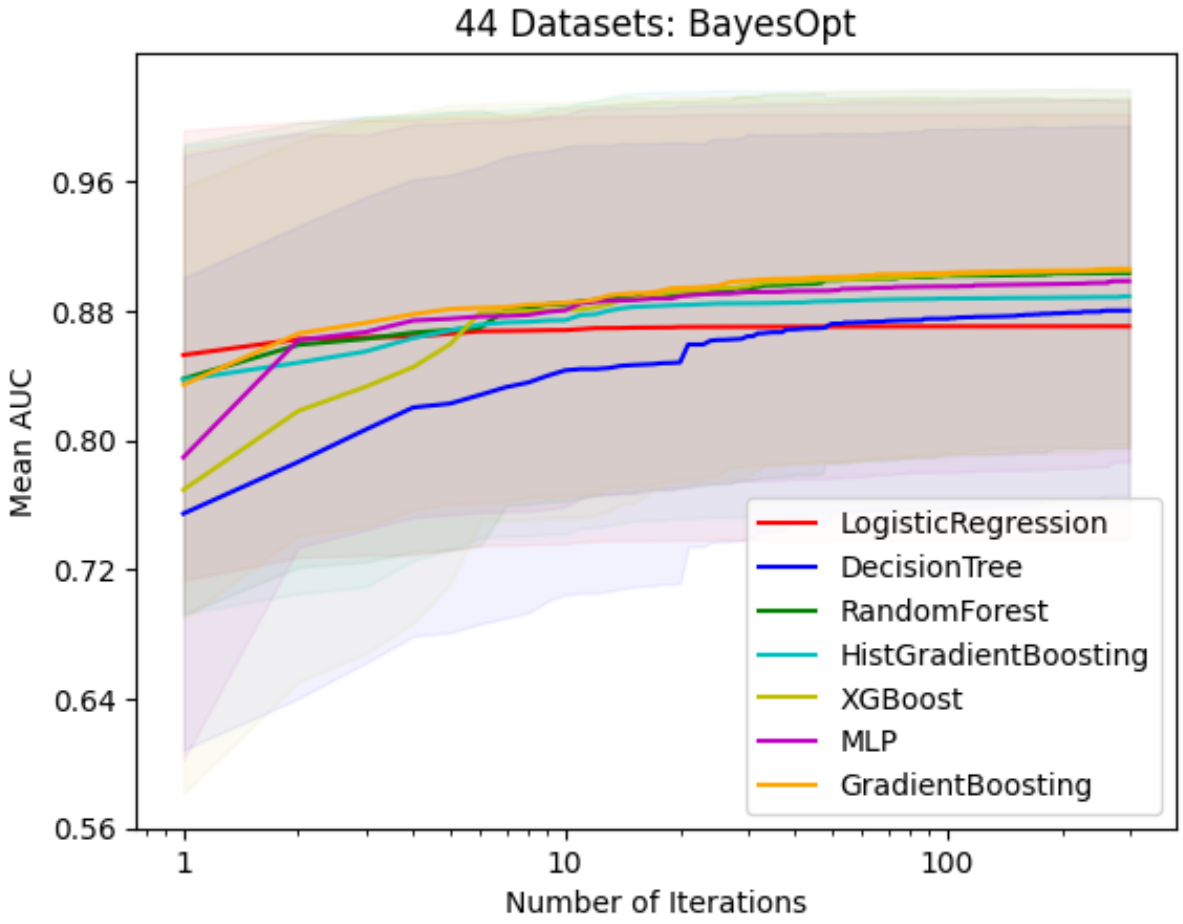

Figure 4.3: Mean AUC of Bayesian optimization results for 44 datasets, with respect to the number of iterations(with shaded +- standard deviation bands)

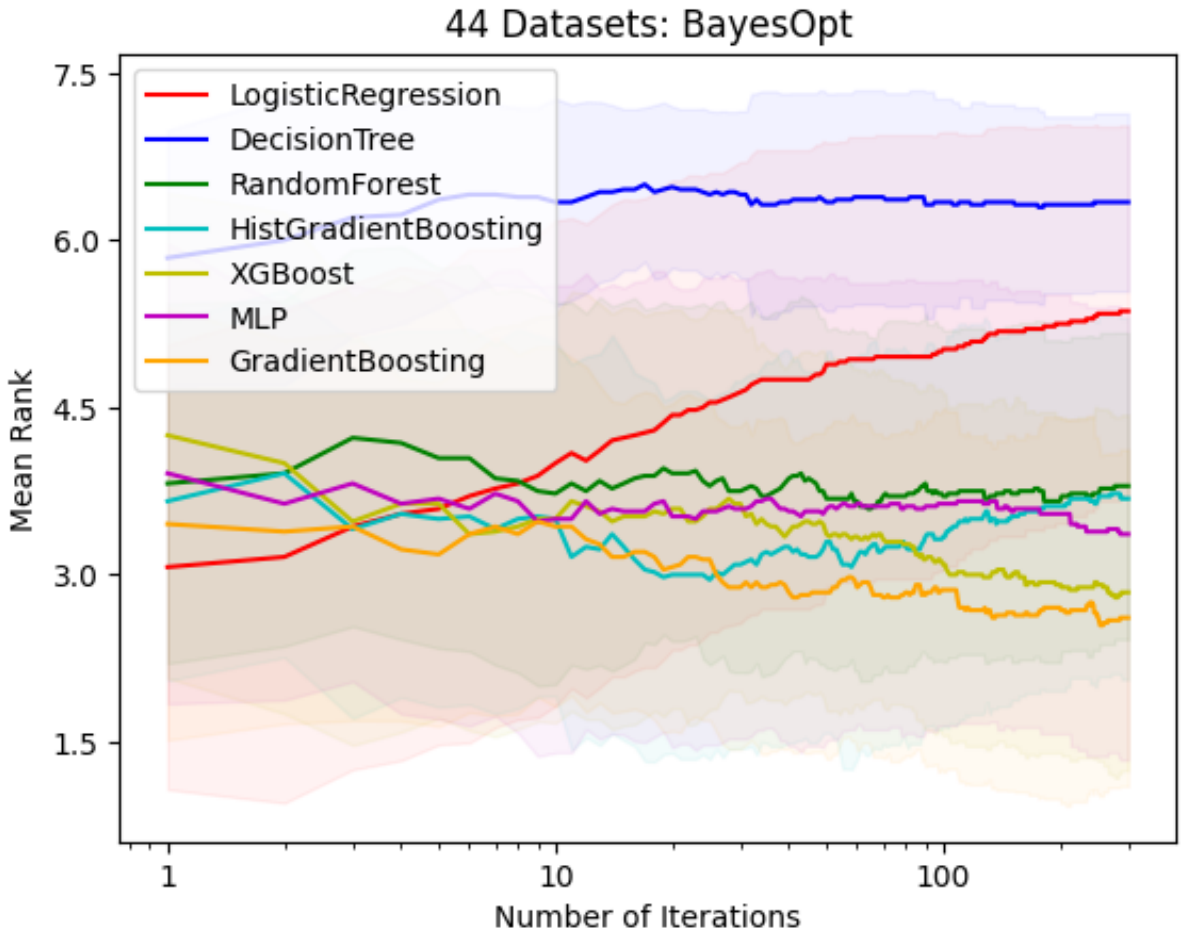

Figure 4.4: Mean rank of Bayesian optimization results for 44 datasets, with respect to the number of iterations(with shaded +- standard deviation bands)

Model	**Mean AUC**	**Mean Rank**
Logistic Regression	0.870+-0.131	5.36+-1.66
Decision Tree	0.880+-0.114	6.34+-0.79
Random Forest	0.903+-0.108	3.79+-1.37
Histogram Gradient Boosting	0.889+-0.128	3.68+-1.62
Gradient Boosting	0.906+-0.110	2.61+-1.51
XGBoost	0.905+-0.107	2.84+-1.59
MLP	0.898+-0.111	3.36+-2.02

Table 4.3: Mean AUC and Rank of 44 Datasets (with +-standard deviation band) at the 300th Iteration of Bayesian optimization

From looking at the mean AUC plot, it is clear that no matter the iteration, trees are marginally better than the DNN MLP as gradient boosting typically has the highest AUC. This superiority margin stays fairly constant throughout all iterations. All the standard deviations of the models appear to be somewhat the same and constant no matter the iteration.

From looking at the mean rank plot, it is clear that no matter the iteration, trees are better than the DNN MLP as at least one tree always has the highest rank. The MLP rank is fairly invariant to hyper-parameter tuning while the majority of trees seem to improve with increased tuning. All the standard deviations of most of the models appear to be somewhat the same and constant no matter the iteration.

All of the highest AUC achieving models have close enough AUC of ~ 0.9 to where you should consider them all as having the same AUC. Meaning that from AUC alone DNNs are just as good as trees. The mean rank plot shows that the tree based model gradient boosting is far superior to the DNN MLP with a difference in rank of 0.75. MLP is not even the 2nd highest ranked model as the tree-based model XGBoost also has a significantly higher rank with a difference of 0.52. Given that Bayesian optimization is also a random process and there are bound to be outliers I would far rather a model with a higher rank than a model that relies on outliers to achieve a high AUC. Therefor for Bayesian optimization given that the rank for 2 tree based models are significantly higher than the DNN MLP, even though the AUC's are the same means that the trees are slightly better than the DNN for the Bayesian optimization benchmark.

The standard deviations for the AUC of the trees and DNN MLP are similar at ~ 0.11 AUC. This is high as $\sim 68.27\%$ (1st standard deviation) of the AUC values for the models are within 44% of the valid 0.5 to 1.0 AUC range. The standard deviations for the rank of the models varies much more than the AUC standard deviations. As the lowest rank standard deviation is for the decision tree with a standard deviation of 0.79. Like with random search this is not surprising as it is a terrible model and is likely always the worst model. The lowest. The DNN MLP has the highest rank standard deviation of 2.02. So the trees rank are more invariant to dataset changes than the DNN MLP rank. Having a reliable rank is obviously preferential to a rank that varies. The average standard deviation for all the models is about 1.5. This is high as $\sim 68.27\%$ (1st standard deviation) of the rank values are within 50% of the valid 1 to 7 rank range.

Now there are the questions of whether hyper-parameter tuning these models using Bayesian optimization was even worth it and if hyper-parameter tuning widens or shrinks the superiority gap of trees with DNNs. Using default models is at least ~ 300 times faster than Bayesian optimization with a budget of 300 iterations, and likely far faster than that given that Bayesian optimization actually has to make decisions unlike random search. The models that achieved $\sim 0.9AUC$ using Bayesian optimization achieved ~ 0.875 AUC using their default models. This

is a pretty significant increase and I would say hyper-parameter tuning is worth it, but once again it is up to whoever is reading this if the time investment is worth it for their application needs.

To answer the second question, the difference in rank between the best DNN and best tree for the default benchmark was 1.57 in favour of the tree. The difference between the best tree and best DNN after being tuned using Bayesian optimization was 0.75 in favour of the tree. This is a substantial difference meaning that hyper-parameter tuning via Bayesian optimization shrinks the superiority gap between trees and DNNs.

4.4 Grid Search

Now that you have seen the effects hyper-parameters have for all the models. I will present the 2D hyper-parameter search spaces for each model and discuss some of the phenomenons that account for the AUC patterns in the plots. I plotted one 2D hyper-parameter search space for each model, where I used the 2 hyper-parameters that had the some of the highest rf importance scores. Grid search, although the least effective HPO algorithm, is quite nice for producing hyper-parameter search spaces as you supply a list of every possible hyper-parameter combination you want plotted. Although these plots do not directly compare trees with DNNs, they do indirectly compare them as it shows how tune-able some of these models are. Also knowing some of the phenomenons behind tree and DNN hyper-parameters is of incredible importance. I implemented grid search using 3 fold stratified cross validation with AUC as the scoring metric and the implementation is in the code snippet A.1.

4.4.1 Logistic Regression

For logistic regression the hyper-parameter search space I plotted is presented in figure 4.5 Where the search space was over hyper-parameters max iter and C. Although it was expected that C was going to be the more important hyper-parameter, I did not expect max iter to have literally no affect whatsoever. This is likely due to the fact that logistic regression always converges in under 50. Logistic regression achieves the best AUC of about 0.865 when C is either 1.62e-01 irrespective of the value of max iter. As with most hyper-parameters it is rare that the search space will be entirely random in that there is typically a pattern like here where there is a gradual change in AUC until the sweet spot is reached. As C is decreased from the sweet spot the AUC decreases significantly down to about 0.830 AUC and as C is increased from the sweet spot the AUC decreases slightly to $\sim$ 0.860 AUC. As C is the inverse of regularization strength when the regularization strength is too low over fitting will occur due to the large weights. When the

regularization strength is too high under fitting occurs in that very low weights are sought after effectively being unable to adequately fit the model.

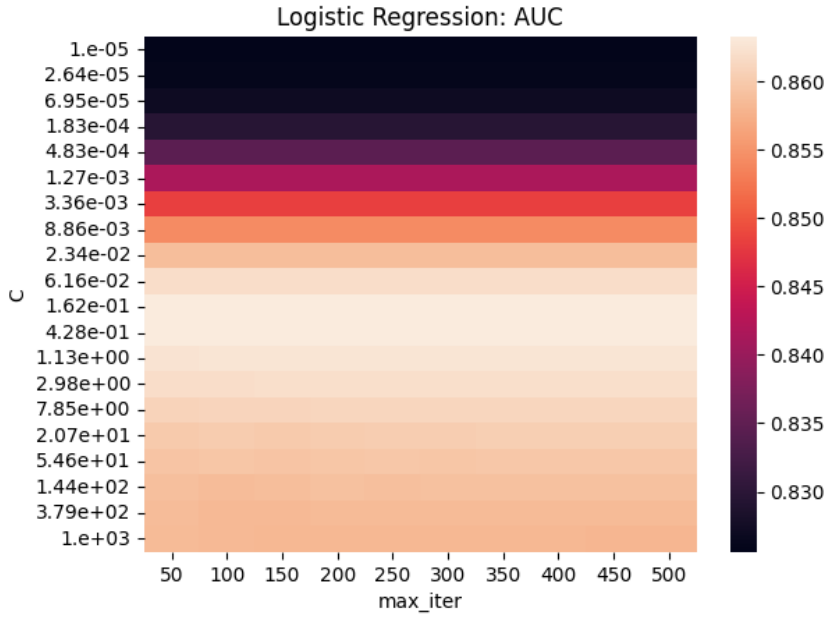

Figure 4.5: Mean AUC achieved by logistic regression on the 44 non-trimmed datasets with respect to hyper-parameters max depth and C.

4.4.2 Decision Tree

For decision tree the hyper-parameter search space I plotted is presented in figure 4.6. Where the search space was over hyper-parameters max depth and min samples leaf. When max depth has a value of 'None' this is equivalent to infinity. So when you increase max depth, the AUC appears to increase as well. This is unexpected as usually decision trees overfit, so regularization tends to be beneficial[27]. The min samples leaf hyper-parameter has a sweet spot between 6 and 31 and decreases in AUC as you move away from this sweet spot. Minimum number of samples in a leaf is a form of regularization and is somewhat similar to max depth in that it restricts the tree from growing. When it is too low little regularization takes place and hence the tree over fits and when it is too high it will not grow and hence underfit resulting in lower AUC.

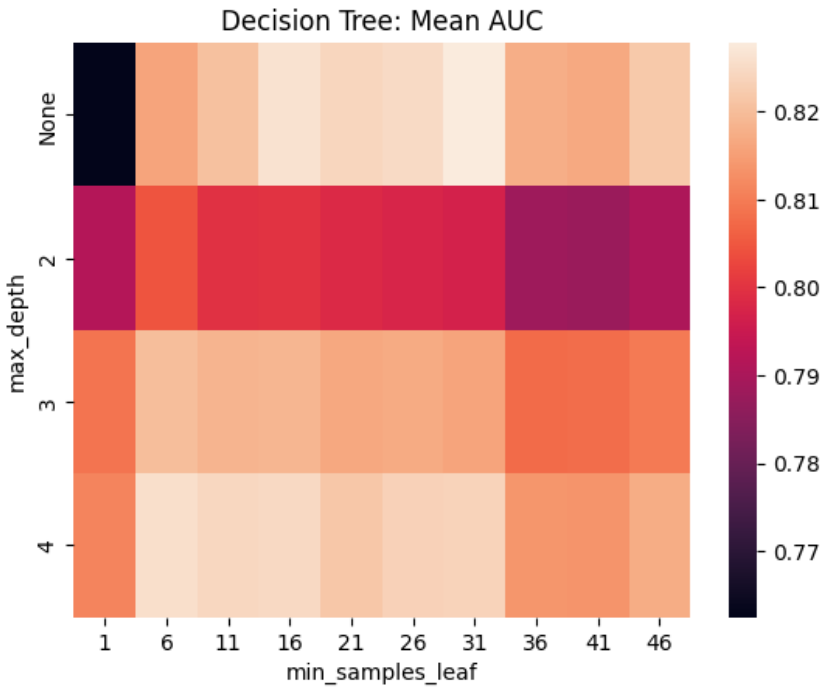

Figure 4.6: Mean AUC achieved by decision tree on the 44 non-trimmed datasets with respect to hyper-parameters max depth and min samples leaf.

4.4.3 Random Forest

For random forest the hyper-parameter search space I plotted is presented in figure 4.7. Where the search space was over hyper-parameters max depth and n estimators. Like decision tree this plot is also affected in a similar way to the max depth. Where an increasing max depth seems to increase AUC. Random forest does not need as aggressive regularization as a decision tree does because it is built-in already[31]. So when increasing max depth results in a higher AUC, this is likely because it does not overfit nearly as much as decision trees and hence benefits from the increased capacity. The n estimators hyper-parameter seems to marginally increase AUC as it increases, with significant diminishing returns past 200 estimators. This is expected as it is tough to overfit a random forest and will benefit from the increased capacity as well.

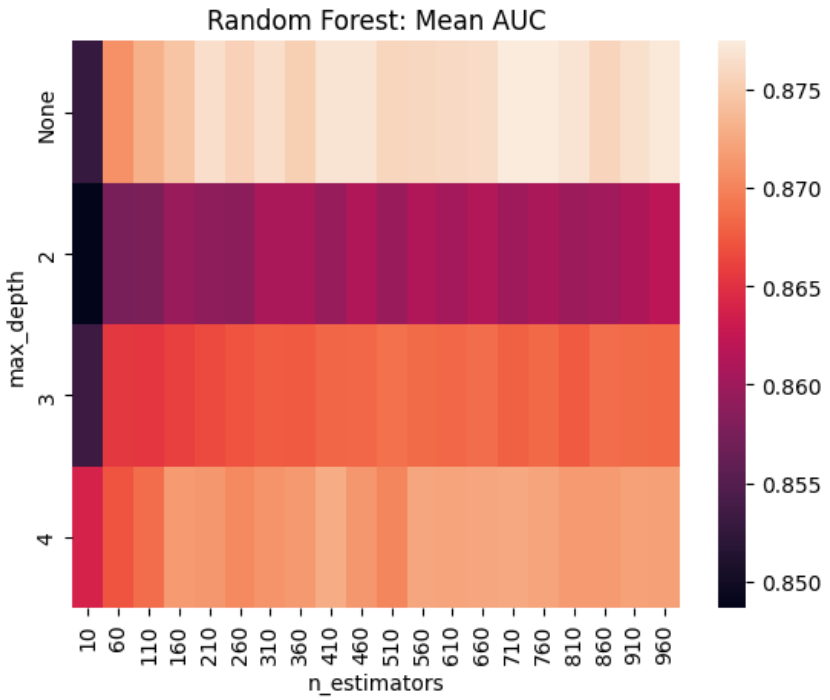

Figure 4.7: Mean AUC achieved by random forest on the 44 non-trimmed datasets with respect to hyper-parameters max depth and n estimators.

4.4.4 Gradient Boosting

For gradient boosting the hyper-parameter search space I plotted is presented in figure 4.8. Where the search space was over hyper-parameters learning rate and the number of estimators. This plot shows that both hyper-parameters play a significant role in the performance of the model. As the lowest AUC of around 0.81 AUC is achieved when they are both small and increases slowly to the highest AUC of 0.88 AUC, when they are both increased. The pattern of this search space where an increasing learning rate results in an increasing score is unexpected as in reality it "is a common pattern that the smaller parameter [learning rate] and therefore, the lower the shrinked boosted increments are, the better generalization is achieved"[32]. Of course if you decrease it so much that the decision trees have no affect whatsoever that would result in worse score, however the smallest learning rate in this plot of 1e-05 is not that extreme of a value[2]. When n estimators is increased the AUC increases this is because it allows to correct the errors of the ensemble even more. Care needs to be taken as increasing this hyper-parameter too far can result in overfitting, however that does not appear to be the case here.

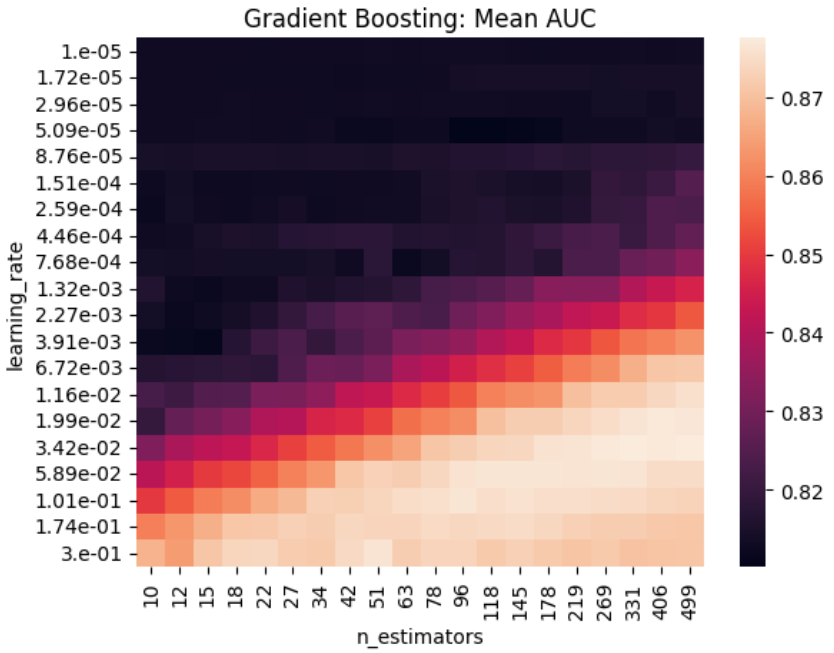

Figure 4.8: Mean AUC achieved by gradient boosting on the 44 non-trimmed datasets with respect to hyper-parameters learning rate and n estimators.

4.4.5 Histogram Gradient Boosting

For histogram gradient boosting the hyper-parameter search space I plotted is presented in figure 4.9. Where the search space was over the hyper-parameters learning rate and the minimum number of samples in a leaf. For this search space the minimum number of samples in a leaf is fairly irrelevant and has little affect on the performance. The learning rate which was expected to be more important achieves the highest AUC of ~ 0.87 at a learning rate of 5.89e-02. As the learning rate decreases the AUC decreases to ~ 0.825 AUC. For the same reasoning with gradient boosting an increasing learning rate resulting in higher AUC is unexpected. The minimum number of samples in a leaf having little affect on the performance is unexpected,however they are also not at extremely high or low values. Minimum number of samples in a leaf is a form of regularization and when you increase it too far you limit the growth of the trees resulting in underfitting and vice versa.

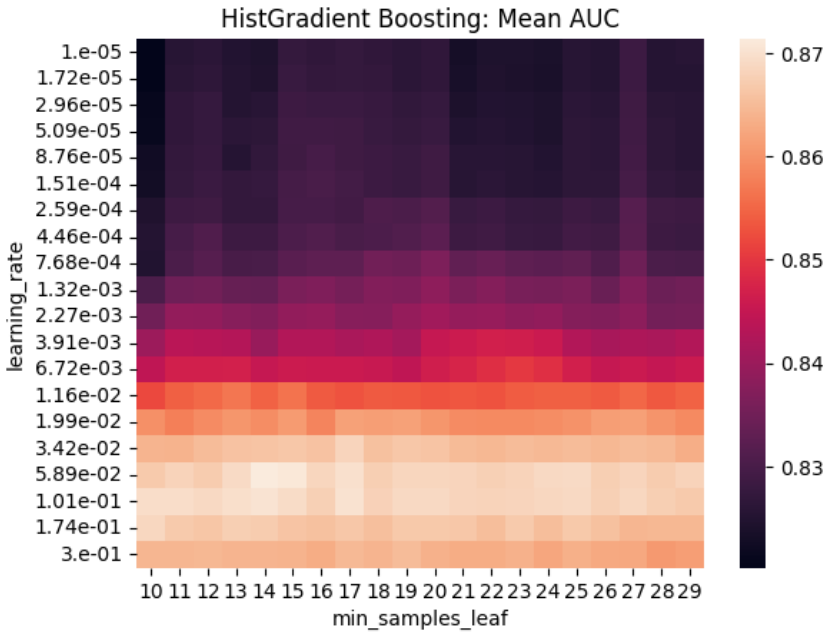

Figure 4.9: Mean AUC achieved by histogram gradient boosting on the 44 non-trimmed datasets with respect to hyper-parameters learning rate and min samples leaf.

4.4.6 XGBoost

For XGBoost the hyper-parameter search space I plotted is presented in figure 4.10. Where the search space was over hyper-parameters learning rate and min child weight. This plot shows how significant tuning hyper-parameters can be,mainly due to min child weight in this case. As when min child weight has a value of 100 the AUC is 0.675 and the optimal configuration is when min child weight is 1 and learning rate is 1.42e-01 leading to an AUC of 0.875 which is a staggering increase of about 0.2 AUC. As the learning rate increases the AUC increases for the same reason as with gradient boosting. As the min child weight increases the AUC decreases. This is because similarly to min samples split the higher the value the less likely a node is to split, resulting in under fitting.

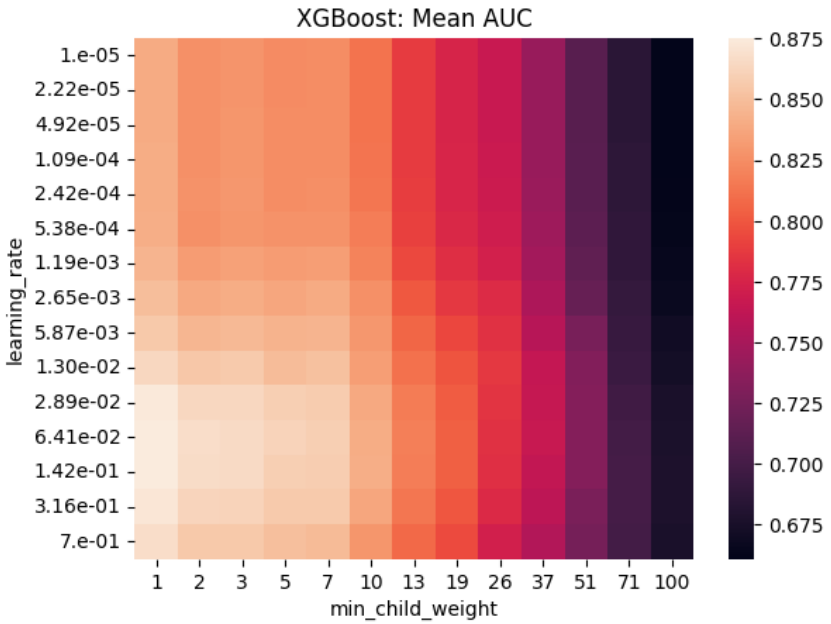

Figure 4.10: Mean AUC achieved by XGBoost on the 44 non-trimmed datasets with respect to hyper-parameters learning rate and min child weight.

4.4.7 MLP

For MLP the hyper-parameter search space I plotted is presented in figure 4.11. Where the search space was over the number of nodes in the first 2 hidden layers. The AUC continuously increases as the size of both hidden layers is increased. It starts with an AUC of about 0.867 AUC when the first and second hidden layers both have 5 nodes and marginally increases to 0.877 AUC when they both have 195 nodes. It is tough to tell whether diminishing returns are happening, however this does appear to be the case. This result is expected as typically when DNNs are "wider or deeper, accuracy on many examples increases"[33], as the model has increased capacity.

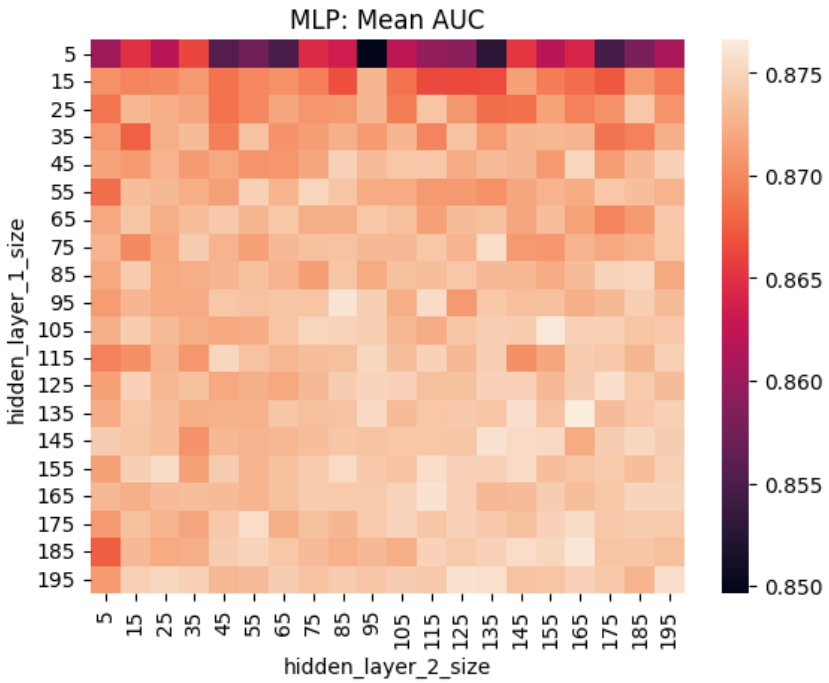

Figure 4.11: Mean AUC achieved by MLP on the 44 non-trimmed datasets with respect to hyper-parameters hidden layer size 1 and hidden layer size 2.

Chapter 5

TabPFN Benchmark Results

These results are all with respect to the 29 datasets that came as a result from the trimming the original 44 datasets. So that the datasets would be appropriate for TabPFN.

5.1 Default Hyper-parameter Models

Looking at table 5.1, it is clear that TabPFN is without a doubt the best model. It achieves an AUC of 0.919 when the best tree being random forest achieves an AUC of 0.907 resulting in a marginal increase of 0.012 AUC. The rank plot shows the absolute superiority of TabPFN with it having the best rank of 2.13 compared to the highest ranked tree being gradient boosting with a rank of 3.79. This is a staggering difference in rank of 1.66 and means the DNN TabPFN is better than all of the trees.

The AUC standard deviations for all the models vary a bit with TabPFN having the lowest standard deviation of 0.081 and logistic regression having the highest standard deviation of 0.134. All of these models have an average sized standard deviation of about 1.0. The rank standard deviations are unsurprising given that many models have similar mean ranks resulting in higher standard deviations for those models and TabPFN having by far the lowest standard deviation of 0.972. As it is typically the best model. Standard deviation is pretty irrelevant in determining which models are best. However, TabPFN having a small AUC/rank standard deviation which results in a reliable AUC/rank irrespective of the dataset further solidifies its place as the best model.

Default Model	Mean AUC	Mean Rank
Logistic Regression	0.870+-0.134	5.72+-2.72
Decision Tree	0.783+-0.121	4.93+-2.43
Random Forest	0.907+-0.0.87	4.17+-2.13
Histogram Gradient Boosting	0.904+-0.091	5.03+-1.49
Gradient Boosting	0.898+-0.094	3.79+-2.24
XGBoost	0.896+-0.093	5.13+-1.81
MLP	0.900+-0.096	5.06+-1.91
TabPFN	0.919+-0.081	2.13+-0.972

Table 5.1: Mean AUC and rank of the default hyper-parameter models, for the 29 trimmed datasets(with shaded +- standard deviation bands)

Model	Mean AUC	Mean Rank
Logistic Regression	0.884+-0.129	5+-2.703
Decision Tree	0.880+-0.104	7.37+-0.761
Random Forest	0.919+-0.081	3.93+-1.76
Histogram Gradient Boosting	0.914+-0.087	4.03+-1.67
Gradient Boosting	0.909+-0.806	5+-1.96
XGBoost	0.914+-0.084	3.96+-1.47
MLP	0.913+-0.09	3.93+-2.33
TabPFN	0.919+-0.081	2.75+-1.94

Table 5.2: Mean AUC and Rank of the 29 trimmed datasets (with +-standard deviation bands) at the 300th iteration of random search

5.2 Random Search

This chapter does random search on the 29 trimmed datasets with the additional TabPFN model and the approach is similar to the previous random search section with the 44 datasets. It is important to remember that TabPFN does not need hyper-parameter tuning and hence its values are from table 5.1 extended for all 300 iterations.

The individual plots are presented in figure C.3 where there is no noteworthy phenomenon that was not already mentioned in the previous chapters.

From looking at the figure 5.2 and 5.1, it is clear that no matter the iteration, TabPFN is by far the best model. This superiority margin shrinks as you allow the other models to be hyper-parameter tuned. All the standard deviations of the models appear to be somewhat the same and constant no matter the iteration. Except the rank standard deviation of TabPFN, which makes sense as you tune the other models while not tuning TabPFN.

Looking at the AUC/rank after significant hyper-parameter tuning at the 300th iteration

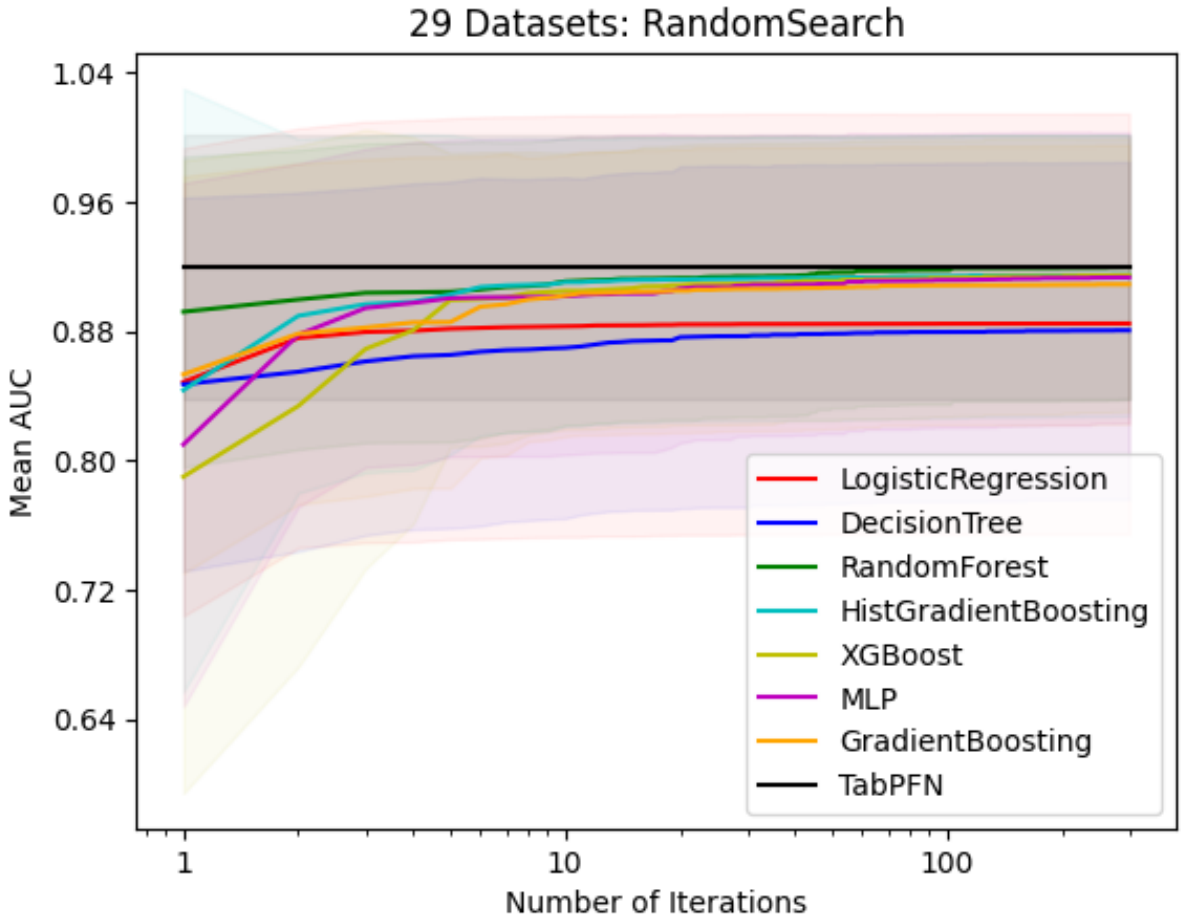

Figure 5.1: Mean AUC of the random search results for the 29 trimmed datasets, with respect to the number of iterations(with shaded +- standard deviation bands)

which is presented in table 5.2, TabPFN and random forest are tied with the highest AUC of 0.919 and the majority of other models getting comparable AUC of $\sim$ 0.910. Looking at the mean rank table after significant hyper-parameter tuning for every model other than TabPFN at the 300th iteration, the obvious superiority of TabPFN although expected is still shocking, having a difference in rank between the 2nd highest ranked models, MLP and Random Forest, of $\sim$ 1.18. With this information combined with the mean AUC information there is absolutely no reason to use any other model.

Now there are the questions of whether hyper-parameter tuning these models using random search was even worth it and if hyper-parameter tuning significantly shrinks the superiority gap of trees with the the DNN TabPFN. The majority of models achieved $\sim$ $0.91AUC$ using random

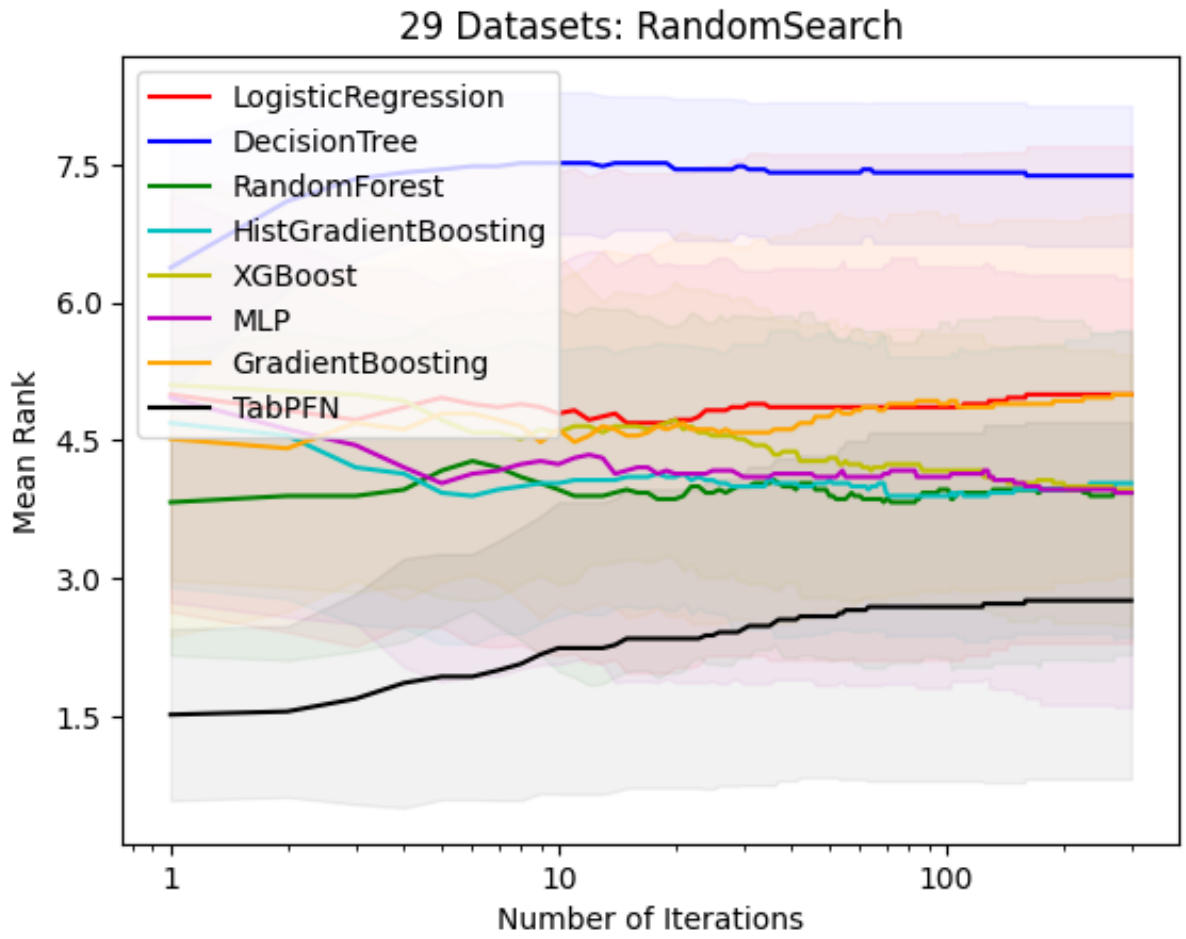

Figure 5.2: Mean rank of the random search results for the 29 trimmed datasets, with respect to the number of iterations(with shaded +- standard deviation bands)

search and achieved $\sim$ 0.9 AUC using their default models. This is a fairly negligible increase and I would say hyper-parameter tuning was not worth it. To answer the second question, the best DNN being TabPFN got a rank 2.13 on the default benchmark whereas the best tree being gradient boosting had a rank of 3.79. This is a difference of 1.66. TabPFN got a rank of 2.75 on the random search benchmark and the best tree being random forest had a rank of 3.93. This is a difference of 1.18 and means that the rank margin decreased from 1.66 on the default benchmark to 1.18 on the random search benchmark. This is a somewhat significant shrinking of the superiority margin between the DNNs and trees, although not as significant of shrinking as I expected to happen.

Chapter 6

Impact and Exploitation

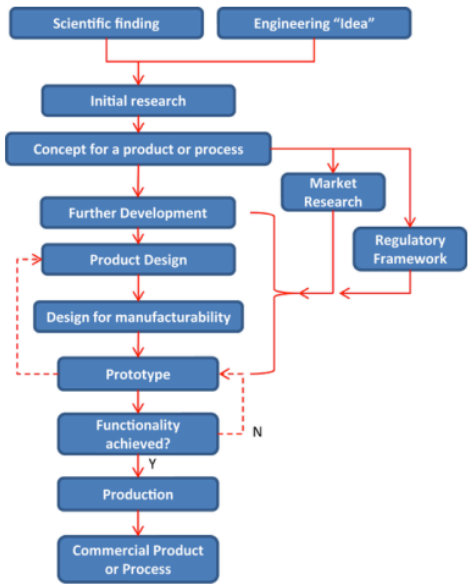

Figure 6.1: A typical block diagram of a product development cycle within the engineering realm[34].

6.1 Research and Development Process

Figure 6.1 depicts a typical block diagram of a product development cycle within the engineering realm. This project undoubtedly fits within the initial research block given that this project is all about researching the pros and cons of trees and DNNs with respect to tabular data. This is

validated as this project compared these two class of models through several benchmarks with findings that show that trees are better than DNNs on the majority of datasets. It shows when using hyper-restricted datasets that the DNN TabPFN is superior to every model.

The research done throughout this project can help to improve an existing application. This application is using ML models to diagnose rare diseases. Where a rare disease is classified as such by the European Union as having no more than 1 case per 2000 people[35]. This leads to incredibly unbalanced datasets of which ML models become incredibly biased towards the non-diagnose class[36]. ML is a good approach for rare-disease exploration as physicians will not know the symptoms of diseases affecting relatively few people.

In the healthcare domain where records are extensively kept, being able to create a dataset of vast size is not a major issue. However even with a dataset of 100,000 samples this leads to the rare-disease class making up at most 50 samples. Another issue is that in many cases we are not allowed to use the various records due to ethical and privacy regulations[37].

So having a ML model that does well on datasets of small size would be very beneficial. As we want to trim all the non-diagnose cases to balance the dataset, which indirectly alleviates privacy concerns. This results in a massive amount of lost data to the non-diagnose cases, however given that this balances the dataset, thereby giving more influence to the diagnose case. The loss of information for the ML models would be made up by the increased balance[36], resulting in a negligible loss in performance.

This is where the research throughout my project comes into play, as TabPFN is a model that is incredibly effective and only effective when the training set contains less than 1000 samples. Given that trimming the severely imbalanced datasets will have negligible performance impact on traditional ML models and we know that TabPFN is better than the tradional ML models on small datasets. This means that TabPFN is better for rare-disease diagnosing than traditional ML models.

To implement this would require having a system in place for collecting data and storing data. To incentivize the public to allow for use of their data will not be an issue either as we do not care if people without a rare-disease allow us to use their data given that the non-diagnose class is so vast. For people that actually have a rare-disease it is likely that it is debilitating and so they will happily share their data in order to treat them. If the rare-disease is not debilitating and they are able to go about their day-to-day without realizing something is wrong than diagnosing them is not a priority either. Of course their are deadly stealth rare-diseases that do not show their symptoms until its too late, however we need to prioritize where the impact will be felt most.

A regulatory framework would need to be put in place to mitigate any privacy and ethical concerns. Given how sensitive some healthcare information can be.

The features that directly impact the ability for TabPFN to classify rare-diseases should be extensively researched. However care needs to be taken in that we do not want 10's of different needles for 10's of features. As the features should be very typical medical data that can be used across all 6000 rare-diseases. Which increases the likeliness of getting data for the non-diagnose class.

Extensive research would need to be done in not just improving TabPFN for rare-disease diagnosing, but in order to get an understanding of the **confusion matrix**. As although not diagnosing someone with a rare-disease when they have it is not as disastrous as with a more common condition like cancer. Given that you were unlikely to be diagnosed for the rare-disease with or without TabPFN, however it is still very bad. An understanding of the confusion matrix would then allow doctors to take the non-diagnose with a grain of salt depending on the false negative rate.

The commercial product of TabPFN for diagnosing rare-diseases could be automatized in that when data is collected. TabPFN is automatically run for every different possible rare-disease and given the speed[9] of TabPFN this could be fed directly back to the doctors computer screen allowing for speedy diagnoses. This involves needing a webpage to run TabPFN which should be relatively easy to implement into their pre-existing web page of where they write their notes for patients.

6.2 Societal Impact

You might question why we should use resources on mitigating the impact of diseases that are rare, as they should affect hardly anyone. However, you are mistaken as "over 6000 distinct rare diseases affect up to 36 million EU citizens"[35], meaning that 12.4% of the 448 million EU citizens are affected by a rare disease. This not only drastically affects their quality of life, but it significantly impacts the economy. As "the overall cost of rare diseases in the U.S. is estimated to be $2.2 trillion per year compared with $3.4 trillion per year for 133 million patients with mass market diseases"[38].

It is primarily the healthcare industry that will be impacted most from this rare-disease diagnosing application and the companies that will benefit most from this application of TabPFN are all the government healthcare authorities like the NHS for the UK.

The research from TabPFN will be able to vastly impact other areas where imbalanced datasets or scarce data is common. This involves credit card fraud,supply chain analysis for smaller startups and anomaly detection. A specific example of anomaly detection is in cyber-security intrusion detection systems where breaches in security is far rarer than normal activity.

Some benefits from this application are less invasion of privacy and being more environemnt-

ally friendly. As TabPFN uses smaller datasets this indirectly results in less invasion of privacy. Both using smaller datasets, and saving time from not needing to hyper-parameter tune or train results in a smaller carbon footprint[9]. There is a caveat, in that TabPFN is pre-trained offline once, however the **amortized** cost of pre-training it offline for "20 hours on one machine with 8 GPUs"[9] once is negligible, when you consider how many datasets TabPFN will be used for. So TabPFN has a lower runtime than other traditional ML models, even though it has high inference time[9].

TabPFN does not only introduce positives unfortunately as there are some negatives to be aware of.As although relatively compared to huge datasets the invasion of privacy is less, this still needs to be approached with care. To alleviate concerns adequate regulations would need to be set in place. Patients should willingly sign a waiver if they want their data to be collected.

The healthcare industry is arguably the industry that most relies on trust. So when some ML models are hard to understand they can erode that trust. TabPFN is not one of the lucky ML models that is interpretable[9] and this is the biggest negative to be aware of with TabPFN.

This means that issues with respect to fairness and accountability come into play. So if TabPFN is unfair towards some demographic you will be unable to detect it. For diagnosing rare-diseases TabPFN can be easily biased. As for example with the UK Biobank[39] the "majority [of the data is for] white [people]"[40]. Where the UK Biobank is a database containing healthcare records on over 500,000 participants for research purposes. Skewing the results of TabPFN towards the white demographic.

Chapter 7

Conclusion

This thesis compared trees with DNNs on tabular data. It did this through several benchmarks on two different sets of datasets. Where the first set contained 44 datasets of medium sample and feature size. Consisting of both categorical and numerical features, where the class distributions were almost perfectly balanced. The second set was a subset of the first set in that it was generated from the first set by deleting every categorical feature and trimming the datasets to less than 1500 samples. This resulted in the set having small sample and feature sizes. Consisting of purely numerical features, where the class distributions were almost perfectly balanced.

The benchmarks generated the AUC and rank for various different trees and DNNs. Where some benchmarks were used to compare the default hyper-parameter models, some compared how an increasing budget to hyper-parameter tuning affected the performance and one plotted the hyper-parameter search spaces.

The default benchmark on the 44 datasets show that trees are marginally better then the DNN. As the best tree achieves a significantly higher rank than the best DNN. The best DNN achieves the same AUC as the best tree.

The random search benchmark with respect to the 44 datasets show that after considerable hyper-parameter tuning the best tree had a marginally higher rank than the best DNN. The best DNN achieves the same AUC as the best tree. It shows that hyper-parameter tuning via random search significantly decreases the superiority rank margin of trees with the DNN.

The Bayesian optimization benchmark with respect to the 44 datasets show that after considerable hyper-parameter tuning the best tree had a marginally higher rank than the best DNN. The best DNN achieves the same AUC as the best tree. It shows that hyper-parameter tuning via Bayesian optimization significantly decreases the superiority rank margin of trees with the DNN.

This thesis shows when using the 29 datasets that the DNN TabPFN is far superior to trees

when comparing default models. It is far superior even after allowing the trees to be considerably hyper-parameter tuned, while itself was not. It shows that hyper-parameter tuning via random search only marginally decreases the superiority margin between trees and the DNN TabPFN.

In conclusion, trees are marginally better than DNNs on the 44 datasets and the DNN TabPFN is far superior to every model on the severely restricted 29 datasets. This thesis has only benchmarked the raw AUC achieved by trees and DNNs, it is up to you to consider the trade-offs with the other side metrics such as interpretability and runtime, for your application needs.

Acknowledgements

I would like to thank my supervisor, Dr Elliot J. Crowley for his immense guidance throughout this project for which this project would not have been possible. I'm incredibly grateful that I got to do a project that I really wanted to do which made this project an absolute joy to work on. I would like to thank my thesis examiner Dr Yunjie Yang for his invaluable feedback throughout this project that allowed me to improve it.
I would like to thank my parents for their enormous support throughout my entire degree. Lastly, I would like to thank my brother who helped me navigate my engineering degree, having graduated himself with an electrical engineering degree a few years ago.

References

[1] Shwartz-Ziv, R. and Armon, A.: 'Tabular data: Deep learning is not all you need,' 2021. [Online]. Available: https://openreview.net/pdf?id=vdgtepS1pV.

[2] Grinsztajn, L., Oyallon, E. and Varoquaux, G.: 'Why do tree-based models still outperform deep learning on tabular data?' 2022. [Online]. Available: http://arxiv.org/abs/2207.08815.

[3] Kadra, A., Lindauer, M., Hutter, F. and Grabocka, J.: 'Well-tuned simple nets excel on tabular datasets,' 2021. [Online]. Available: http://arxiv.org/abs/2106.11189.

[4] en. [Online]. Available: https://scikit-learn.org/stable/.

[5] en. [Online]. Available: https://xgboost.readthedocs.io/en/stable/.

[6] en. [Online]. Available: https://pytorch.org/docs/stable/index.html.

[7] en. [Online]. Available: https://colab.research.google.com/.

[8] Turner, R., Eriksson, D., McCourt, M. *et al.*: 'Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020,' 2021. [Online]. Available: http://arxiv.org/abs/2104.10201.

[9] Hollmann, N., Müller, S., Eggensperger, K. and Hutter, F., 'TabPFN: A transformer that solves small tabular classification problems in a second,' Jul. 2022. arXiv: 2207.01848 [cs.LG].

[10] Szeghalmy, S. and Fazekas, A., 'A comparative study of the use of stratified cross-validation and distribution-balanced stratified cross-validation in imbalanced learning,' en, *Sensors (Basel, Switzerland)*, vol. 23, no. 4, p. 2333, 2023, ISSN: 1424-8220. DOI: 10.3390/s23042333. [Online]. Available: http://dx.doi.org/10.3390/s23042333.

[11] en. [Online]. Available: https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc.

[12] contributors, W.: *Who wants to be a millionaire?* Apr. 2024. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Who_Wants_to_Be_a_Millionaire%3F&oldid=1220286369.

[13] Ke, G., Meng, Q., Finley, T. *et al.*: *Lightgbm: A highly efficient gradient boosting decision tree.* [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf.

[14] Chen, T. and Guestrin, C.: *Xgboost: A scalable tree boosting system.* [Online]. Available: http://arxiv.org/abs/1603.02754.

[15] Linardatos, P., Papastefanopoulos, V. and Kotsiantis, S., 'Explainable ai: A review of machine learning interpretability methods,' en, *Entropy (Basel, Switzerland)*, vol. 23, no. 1, p. 18, 2020, ISSN: 1099-4300. DOI: 10.3390/e23010018. [Online]. Available: http://dx.doi.org/10.3390/e23010018.

[16] en. [Online]. Available: https://scikit-optimize.github.io/stable/.

[17] Frazier, P. I.: *A tutorial on bayesian optimization.* [Online]. Available: http://arxiv.org/abs/1807.02811.

[18] Raschka, S.: *A short chronology of deep learning for tabular data*, en, Jul. 2022. [Online]. Available: https://sebastianraschka.com/blog/2022/deep-learning-for-tabular-data.html.

[19] [Online]. Available: https://www.kaggle.com/.

[20] *UCI machine learning repository*, en, https://archive.ics.uci.edu/datasets, Accessed: 2024-4-16.

[21] *Murat koklu datasets*, en, https://www.muratkoklu.com/datasets/, Accessed: 2024-4-16.

[22] *AI Mathematical Olympiad - Progress Prize 1 — kaggle.com*, https://www.kaggle.com/competitions/ai-mathematical-olympiad-prize, [Accessed 19-04-2024].

[23] *Home Credit - Credit Risk Model Stability — kaggle.com*, https://www.kaggle.com/competitions/home-credit-credit-risk-model-stability, [Accessed 19-04-2024].

[24] 'Body area networks: Smart IoT and big data for intelligent health management: 14Th EAI international conference, BODYNETS 2019, Florence, Italy, October 2-3, 2019, proceedings'. (Springer International Publishing, 2019), ISBN: 9783030348328.

[25] [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

[26] [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html.

[27] Leiva, R. G., Anta, A. F., Mancuso, V. and Casari, P., 'A novel hyperparameter-free approach to decision tree construction that avoids overfitting by design,' DOI: 10.1109/ACCESS.201x.xxxxxxx. [Online]. Available: http://arxiv.org/abs/1906.01246.

[28] Corbacioglu, Ş. and Aksel, G., 'Receiver operating characteristic curve analysis in diagnostic accuracy studies: A guide to interpreting the area under the curve value,' en, *Turkish journal of emergency medicine*, vol. 23, no. 4, p. 195, 2023, ISSN: 2452-2473. DOI: 10.4103/tjem.tjem_182_23. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/38024184/.

[29] Bhowmik, R. and Wang, S., 'Stock market volatility and return analysis: A systematic literature review,' en, *Entropy (Basel, Switzerland)*, vol. 22, no. 5, p. 522, 2020, ISSN: 1099-4300. DOI: 10.3390/e22050522. [Online]. Available: http://dx.doi.org/10.3390/e22050522.

[30] *Hyperopt: A Python library for model selection and hyperparameter optimization - IOPscience — iopscience.iop.org*, https://iopscience.iop.org/article/10.1088/1749-4699/8/1/014008/meta, [Accessed 20-04-2024].

[31] Zhou, S.: *Random forests and regularization.* [Online]. Available: https://d-scholarship.pitt.edu/43285/1/ETD_Siyu_Zhou%2014%20July_1.pdf.

[32] Natekin, A. and Knoll, A., 'Gradient boosting machines, a tutorial,' DOI: 10.3389/fnbot.2013.00021/abstract. [Online]. Available: http://dx.doi.org/10.3389/fnbot.2013.00021/abstract.

[33] Nguyen, T., Raghu, M. and Kornblith, S.: *Do wide and deep networks learn the same things? uncovering how neural network representations vary with width and depth.* [Online]. Available: http://arxiv.org/abs/2010.15327.

[34] en. [Online]. Available: https://www.ukri.org/.

[35] en. [Online]. Available: https://research-and-innovation.ec.europa.eu/research-area/health/rare-diseases_en.

[36] Johnson, J. M. and Khoshgoftaar, T. M., 'Survey on deep learning with class imbalance,' en, *Journal of big data*, vol. 6, no. 1, 2019, ISSN: 2196-1115. DOI: 10.1186/s40537-019-0192-5. [Online]. Available: http://dx.doi.org/10.1186/s40537-019-0192-5.

[37] Yadav, N., Pandey, S., Gupta, A., Dudani, P., Gupta, S. and Rangarajan, K., 'Data privacy in healthcare: In the era of artificial intelligence,' en, *Indian dermatology online journal*, vol. 14, no. 6, pp. 788–792, 2023, ISSN: 2229-5178. DOI: 10.4103/idoj.idoj_543_23. [Online]. Available: http://dx.doi.org/10.4103/idoj.idoj_543_23.

[38] Andreu, P., Jenny, K. P. D., Child, C., Mba, G. C. and Jd, G. C.: *The burden of rare diseases: An economic evaluation.* [Online]. Available: https://chiesirarediseases.com/assets/pdf/chiesiglobalrarediseases.whitepaper-feb.-2022_production-proof.pdf.

[39] en. [Online]. Available: https://www.ukbiobank.ac.uk/.

[40] Yang, L. and Altman, R.: *Using machine learning to predict rare diseases*, en. [Online]. Available: https://hai.stanford.edu/news/using-machine-learning-predict-rare-diseases.

[41] en. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[42] en. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html.

[43] en. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html.

[44] en. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html.

[45] en. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html.

[46] en. [Online]. Available: https://xgboost.readthedocs.io/en/latest/parameter.html.

Appendix A

Code

```
#generated from colab notebook
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_datasets = 0
n_iter = 300
cv = 3
def float_to_int(rvs):
    def rvs_wrapper(*args, **kwargs):
        return rvs(*args, **kwargs).round().astype(int)
    return rvs_wrapper
def int_loguniform(low, high):
    lu = loguniform(low, high)
    lu.rvs = float_to_int(lu.rvs)
    return lu
def int_normal(mean, sigma):
    n = norm(loc = mean, scale = sigma)
    n.rvs = float_to_int(n.rvs)
    return n
def isBinary(series):
    vals = (np.unique(series))
    if len(vals) == 2:
        return True
def getCategoricalColumnsNames(df):
    listCatColumns = []
    for column in df:
        if df[column].dtype == 'object' or isBinary(df[column]) or df[column].
            dtype == 'bool' or df[column].dtype == 'string' or df[column].dtype
            == 'category':
             listCatColumns.append(column)
    return listCatColumns
def doRandomSearch(df,filename):
```

```
global num_datasets
global modelsAccuracyAvgAllDatasetsAllIters
global n_iter
global cv
classEncoder = LabelEncoder()
features = df.columns.values.tolist()
features = [value for value in features if value != 'Class']
X = df.loc[:, features]
y = df['Class'].to_numpy()
categoricalColumns = getCategoricalColumnsNames(X)
columnsToStandardize = X.columns.difference(categoricalColumns)
X = pd.get_dummies(data=X, columns=categoricalColumns,drop_first=True)
ct = ColumnTransformer([
    ('somename', StandardScaler(), [X.columns.get_loc(c) for c in
        columnsToStandardize])
], remainder='passthrough')
y = classEncoder.fit_transform(y)
models = [0, 0, 0, 0, 0, 0,0]
models[0] = LogisticRegression()
models[1] = DecisionTreeClassifier()
models[2] = RandomForestClassifier()
models[3] = HistGradientBoostingClassifier()
models[4] = GradientBoostingClassifier()
models[5] = xgb.XGBClassifier(
                            tree_method='gpu_hist',
                            predictor='gpu_predictor',
                            gpu_id=0,
                            )
models[6] = NeuralNetClassifier(
                  module=PyTorchMLP,
                  module__num_layers = 2,
                  module__layer1 = 100,
                  module__layer2 = 100,
                  module__layer3 = 100,
                  module__layer4 = 100,
                  module__layer5 = 100,
                  module__input_dim=X.shape[1],
                  module__output_dim=2,
                  criterion=torch.nn.CrossEntropyLoss,
                  optimizer=torch.optim.Adam,
                  optimizer__weight_decay=0.0001,
                  batch_size=200,
                  lr=0.001,
                  max_epochs=200,
```

```
                    iterator_train__shuffle=True,
                    callbacks=[EarlyStopping(patience=10)],
                    verbose = 0,
                    device='cuda' if torch.cuda.is_available() else 'cpu',
                    )
for (model_idx,model) in enumerate(models):
    model_name = ''
    scorer = 'roc_auc'
    match model:
        case LogisticRegression():
            param_grid = {'estimator__C': loguniform(1e-6, 1000)}
            model_name = 'LogisticRegression'
        case DecisionTreeClassifier():
            param_grid = {
                        'estimator__max_depth': [None,2,3,4],
                        'estimator__max_leaf_nodes': int_normal(31, 5),
                        'estimator__min_samples_leaf': int_loguniform
                            (1,50)}
            model_name = 'DecisionTree'
        case RandomForestClassifier():
            param_grid = {
                        'estimator__n_estimators':int_loguniform(10,1000),
                        'estimator__max_depth': [None,2,3,4],
                        'estimator__min_samples_leaf': int_loguniform
                            (1,50),
                        'estimator__max_features':['sqrt', 'log2', None,
                            0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]}
            model_name = 'RandomForest'
        case HistGradientBoostingClassifier():
            param_grid = {'estimator__learning_rate':lognorm(scale = (0.01),
                s=np.log(10)),
                       'estimator__min_samples_leaf': int_normal(20,2),
                       'estimator__max_leaf_nodes': int_normal(31, 5),
                         'estimator__max_depth': [None,2,3,4]
                         }
            model_name = 'HistGradientBoosting'
        case GradientBoostingClassifier():
            param_grid = {'estimator__learning_rate':lognorm(scale = (0.01),
                s=np.log(10)),
                        'estimator__n_estimators':int_loguniform(10,500),
                        'estimator__max_depth': [None,2,3,4,5]
                        }
            model_name = 'GradientBoosting'
        case xgb.XGBClassifier():
```

```
        param_grid = {'estimator__max_depth': list(range(1,11)),
                      'estimator__learning_rate': loguniform(1e-5, 0.7),
                      'estimator__min_child_weight': int_loguniform(1,1
                          e2),
                      'estimator__reg_alpha': loguniform(1e-8, 1e2),
                      'estimator__colsample_bytree' : uniform(0.5,0.5),
                      'estimator__subsample':uniform(0.5,0.5),
                      'estimator__n_estimators':int_loguniform(90,1000)}
        model_name = 'XGBoost'
    case NeuralNetClassifier():
        param_grid = {
                      'estimator__module__num_layers': [1,2,3],
                      'estimator__module__layer1': list(range(10, 251)),
                      'estimator__module__layer2': list(range(10, 251)),
                      'estimator__module__layer3': list(range(10, 251)),
                      'estimator__module__layer4': list(range(10, 251)),
                      'estimator__module__layer5': list(range(10, 251)),
                      'estimator__optimizer__weight_decay': loguniform(1
                          e-5, 0.1),
                      'estimator__lr': loguniform(1e-6, 1e-2),
                      'estimator__batch_size': list(range(64, 513))
                      }
        model_name = 'MLP'
        scorer = make_scorer(roc_auc_score, greater_is_better=True,
            needs_proba=True)
pipeline_tmp = Pipeline(steps = [
('column_transformer', ct)])
if ((model_idx) != 6 and str(model) != str(LogisticRegression())):
    pipeline_tmp = Pipeline(steps=[])
    pipeline_tmp.steps.append(['estimator',model])
rs = RandomizedSearchCV(estimator=pipeline_tmp,
                        n_iter= n_iter,
                        param_distributions=param_grid,
                        scoring=scorer,
                        cv=cv,
                        refit=False,
                        )
rs = rs.fit(X.to_numpy().astype(np.float32), y)
model_randSearch_out_df = pd.DataFrame.from_dict(rs.cv_results_)
out_path = '/content/drive/MyDrive/Colab␣Data/Organized/Data␣Results␣
    Ultimate/RandomSearch/'
model_randSearch_out_df.to_csv(out_path+str(filename.split('.')[0])+'/'
    + model_name+ '.csv')
```

```
class PyTorchMLP(nn.Module):
    def __init__(self, input_dim, num_layers=3,layer1=100,layer2=100,layer3=100,
        layer4=100,layer5=100, output_dim=2, nonlin=nn.ReLU()):
        super(PyTorchMLP, self).__init__()
        hidden_layer_sizes = (layer1,layer2,layer3,layer4,layer5)
        hidden_layer_sizes = hidden_layer_sizes[:num_layers]
        layers = []
        layers.append(nn.Linear(input_dim, hidden_layer_sizes[0]))
        layers.append(nonlin)
        for i in range(1, len(hidden_layer_sizes)):
            layers.append(nn.Linear(hidden_layer_sizes[i-1], hidden_layer_sizes[
                i]))
            layers.append(nonlin)
        layers.append(nn.Linear(hidden_layer_sizes[-1], output_dim))
        self.model = nn.Sequential(*layers)
    def forward(self, X):
        return self.model(X)

num_datasets=0
directory = '/content/drive/MyDrive/Colab␣Data/Organized/RunnableDatasets␣
    Ultimate/All␣Datasets'
for filename in os.listdir(directory):
    f = os.path.join(directory, filename)
    if os.path.isfile(f) and filename.split('.')[1]=='csv':
        df = pd.read_csv(f)
        doRandomSearch(df,filename)
```

Listing A.1: Code to run random search benchmark. To run the 29 trimmed datasets adjust the directory=path variable. Disclaimer: I have trimmed many things(only affecting readability of code) and imports so that it would fit in the 30 page appendix. Disclaimer: The very small and irrelevant float to int wrapper methods(floattoint(),intloguniform(),intnormal()) I copied from stack overflow to allow for int distributions as opposed to float distributions. Disclaimer: I have not included code for the default benchmark as there is no room(as it needs to be less than 30 pages) and it is simple to modify this code to get the default benchmark,same for grid search. I have not included code to plot either as there is no room.

```
#Automatically generated by Colab.
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_datasets = 0
n_iter = 300
cv = 3
def isBinary(series):
    vals = (np.unique(series))
```

```
    if len(vals) == 2:
        return True
def getCategoricalColumnsNames(df):
    listCatColumns = []
    for column in df:
        if df[column].dtype == 'object' or isBinary(df[column]) or df[column].
            dtype == 'bool' or df[column].dtype == 'string' or df[column].dtype
            == 'category':
             listCatColumns.append(column)
    return listCatColumns
def doRandomSearch(df,filename):
    global num_datasets
    global modelsAccuracyAvgAllDatasetsAllIters
    global n_iter
    global cv
    classEncoder = LabelEncoder()
    features = df.columns.values.tolist()
    features = [value for value in features if value != 'Class']
    X = df.loc[:, features]
    y = df['Class'].to_numpy()
    categoricalColumns = getCategoricalColumnsNames(X)
    columnsToStandardize = X.columns.difference(categoricalColumns)
    X = pd.get_dummies(data=X, columns=categoricalColumns,drop_first=True)
    ct = ColumnTransformer([
        ('somename', StandardScaler(), [X.columns.get_loc(c) for c in
            columnsToStandardize])
    ], remainder='passthrough')
    y = classEncoder.fit_transform(y)
    models = [0, 0, 0, 0, 0, 0,0]
    models[0] = LogisticRegression()
    models[1] = DecisionTreeClassifier()
    models[2] = RandomForestClassifier()
    models[3] = HistGradientBoostingClassifier()
    models[4] = GradientBoostingClassifier()
    models[5] = xgb.XGBClassifier(
                                  tree_method='gpu_hist',
                                  predictor='gpu_predictor',
                                  gpu_id=0,
                                  )
    models[6] = NeuralNetClassifier(
                        module=PyTorchMLP,
                        module__nonlin=nn.ReLU(),
                        module__num_layers = 3,
                        module__layer1 = 100,
```

```
                    module__layer2 = 100,
                    module__layer3 = 100,
                    module__layer4 = 100,
                    module__layer5 = 100,
                    module__input_dim=X.shape[1],
                    module__output_dim=2,
                    criterion=torch.nn.CrossEntropyLoss(),
                    optimizer=torch.optim.Adam,
                    optimizer__weight_decay=0.0001,
                    optimizer__eps=1e-8,
                    optimizer__betas=(0.9, 0.999),
                    batch_size=200,
                    lr=0.001,
                    max_epochs=200,
                    iterator_train__shuffle=True,
                    callbacks=[EarlyStopping(patience=10)],
                    verbose = 0,
                    device='cuda' if torch.cuda.is_available() else 'cpu',
                    )
for (model_idx,model) in enumerate(models):
    model_name = ''
    scorer = 'roc_auc'
    param_grid_all = {}
    match model:
        case LogisticRegression():
            param_grid = {'estimator__C': Real(1e-5, 5,prior='log-uniform'),
                          'estimator__max_iter': Integer(50,500,prior='
                              uniform'),
                          'estimator__fit_intercept': Categorical([True,
                              False]),
                          'estimator__penalty': Categorical(['l2',None])}
            model_name = 'LogisticRegression'
        case DecisionTreeClassifier():
            param_grid = {
                          'estimator__ccp_alpha' : Real(0.0001, 0.1,prior='
                              log-uniform'),
                          'estimator__max_depth': Integer(1,30,prior='
                              uniform'),
                          'estimator__min_samples_leaf': Integer(1,50,prior=
                              'uniform'),
                          'estimator__min_samples_split':Integer(2,50,prior=
                              'uniform'),
                          'estimator__max_features':Categorical(['sqrt', '
                              log2', None, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6,
```

```
                    0.7, 0.8, 0.9]),
                'estimator__criterion':Categorical(['gini','
                    entropy']),
                'estimator__min_impurity_decrease':Categorical
                    ([0.0,0.1e-7,1e-6,1e-5,1e-4,1e-3,1e-2])
                }
    model_name = 'DecisionTree'
case RandomForestClassifier():
    param_grid = {
                'estimator__n_estimators':Integer(10, 3000, prior=
                    'log-uniform'),
                'estimator__max_depth': Categorical([None,2,3,4]),
                'estimator__min_samples_leaf': Integer(1, 51,
                    prior='log-uniform'),
                'estimator__max_features':Categorical(['sqrt', '
                    log2', None, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6,
                    0.7, 0.8, 0.9]),
                'estimator__min_impurity_decrease': Categorical
                    ([0.0,0.01,0.02,0.05]),
                'estimator__bootstrap':Categorical([True,False]),
                'estimator__criterion':Categorical(['gini','
                    entropy']),
                'estimator__min_samples_split':Categorical([2,3])
                }
    model_name = 'RandomForest'
case HistGradientBoostingClassifier():
    param_grid = {'estimator__learning_rate':Real(1e-6, 0.3,prior='
        log-uniform'),
                'estimator__min_samples_leaf': Integer(16, 24, prior
                    ='uniform'),
                'estimator__max_leaf_nodes': Integer(22, 40, prior='
                    uniform'),
                 'estimator__max_depth': Categorical([None,2,3,4]),
                 'estimator__l2_regularization': Real(1e-5,0.1,
                    prior='log-uniform'),
                 'estimator__max_iter':Integer(50,200,prior='
                    uniform'),
                 'estimator__max_bins': Integer(100,255,prior='
                    uniform')
                 }
    model_name = 'HistGradientBoosting'
case GradientBoostingClassifier():
    param_grid = {'estimator__learning_rate':Real(1e-6, 0.3,prior='
        log-uniform'),
```

```
                    'estimator__min_samples_leaf': Integer(1,50,prior=
                        'log-uniform'),
                    'estimator__max_depth': Categorical([None
                        ,2,3,4,5]),
                    'estimator__loss': Categorical(['deviance','
                        exponential']),
                    'estimator__subsample': Real(0.5,1,prior='uniform'
                        ),
                    'estimator__n_estimators':Integer(10,1000,prior='
                        log-uniform'),
                    'estimator__criterion':Categorical(['friedman_mse'
                        ,'squared_error']),
                    'estimator__min_samples_split':Integer(2,3,prior='
                        uniform'),
                    'estimator__max_leaf_nodes':Categorical([None
                        ,5,10,15]),
                    'estimator__min_impurity_decrease':Categorical
                        ([0.0,0.01,0.02,0.05])
                    }
    model_name = 'GradientBoosting'
case xgb.XGBClassifier():
    param_grid = {
                      'estimator__max_depth': Integer(1, 11, prior='
                          uniform'),
                      'estimator__learning_rate': Real(1e-5, 0.7,
                          prior='log-uniform'),
                      'estimator__min_child_weight': Integer(1, 1e2,
                          prior='log-uniform'),
                      'estimator__reg_alpha': Real(1e-8, 1e2, prior='
                          log-uniform'),
                      'estimator__colsample_bytree' : Real(0.5,1,
                          prior='uniform'),
                      'estimator__subsample':Real(0.5, 1, prior='
                          uniform'),
                      'estimator__n_estimators':Integer(90, 1000,
                          prior='log-uniform'),
                      'estimator__gamma': Real(1e-8, 7, prior='log-
                          uniform'),
                      'estimator__reg_lambda': Real(1, 4, prior='log-
                          uniform'),
                      'estimator__colsample_bylevel':Real(0.5,1, prior
                          ='uniform'),
                      'estimator__objective': Categorical(['binary:
                          logistic','binary:logitraw','binary:hinge'])
```

```
                                ,
                               }
                model_name = 'XGBoost'
            case NeuralNetClassifier():
                param_grid = {
                    'estimator__module__nonlin':  Categorical([nn.Identity(), nn
                        .Sigmoid(), nn.Tanh(),nn.ReLU()]),
                    'estimator__lr': Real(1e-5, 1e-2, prior='log-uniform'),
                    'estimator__optimizer__weight_decay': Real(1e-6, 1, prior='
                        log-uniform'),
                    'estimator__batch_size': Integer(64, 512),
                    'estimator__module__num_layers': Integer(1, 5),
                    'estimator__module__layer1': Integer(1, 200),
                    'estimator__module__layer2': Integer(1, 200),
                    'estimator__module__layer3': Integer(1, 200),
                    'estimator__module__layer4': Integer(1, 200),
                    'estimator__module__layer5': Integer(1, 200),
                    'estimator__max_epochs': Integer(50,500),
                    'estimator__optimizer__eps':Real(1e-10, 1e-6, prior='log-
                        uniform')
                }
                model_name = 'MLP'
                scorer = make_scorer(roc_auc_score, greater_is_better=True,
                    needs_proba=True)
        pipeline_tmp = Pipeline(steps = [
        ('column_transformer', ct)])
        if ((model_idx) != 6 and str(model) != str(LogisticRegression())):
            pipeline_tmp = Pipeline(steps=[])
        pipeline_tmp.steps.append(['estimator',model])
        rs = BayesSearchCV(estimator=pipeline_tmp,
                                n_iter= n_iter,
                                search_spaces=param_grid,
                                scoring=scorer,
                                cv=cv,
                                refit=False,
                                )
        np.int = int #error if you don't do this
        rs = rs.fit(X.to_numpy().astype(np.float32), y)
        model_randSearch_out_df = pd.DataFrame.from_dict(rs.cv_results_)
        out_path = '/content/drive/MyDrive/Colab␣Data/Organized/Data␣Results␣
            Ultimate/BayesOpt/'
        model_randSearch_out_df.to_csv(out_path+str(filename.split('.')[0])+'/'
            + model_name+ '.csv')
class PyTorchMLP(nn.Module):
```

```
    def __init__(self, input_dim, num_layers=3,layer1=100,layer2=100,layer3=100,
        layer4=100,layer5=100, output_dim=2, nonlin=nn.ReLU()):
        super(PyTorchMLP, self).__init__()
        hidden_layer_sizes = (layer1,layer2,layer3,layer4,layer5)
        hidden_layer_sizes = hidden_layer_sizes[:num_layers]
        layers = []
        layers.append(nn.Linear(input_dim, hidden_layer_sizes[0]))
        layers.append(nonlin)
        for i in range(1, len(hidden_layer_sizes)):
            layers.append(nn.Linear(hidden_layer_sizes[i-1], hidden_layer_sizes[
                i]))
            layers.append(nonlin)
        layers.append(nn.Linear(hidden_layer_sizes[-1], output_dim))
        self.model = nn.Sequential(*layers)
    def forward(self, X):
        return self.model(X)
num_datasets=0
directory = '/content/drive/MyDrive/Colab␣Data/Organized/RunnableDatasets␣
    Ultimate/All␣Datasets'
for filename in (os.listdir(directory)):
    f = os.path.join(directory, filename)
    if os.path.isfile(f) and filename.split('.')[1]=='csv' and (filename in
        list_files_4th):
        df = pd.read_csv(f)
        doRandomSearch(df,filename)
```

Listing A.2: Code to run Bayesian optimization benchmark. Disclaimer: I have trimmed many things(only affecting readability of code) and imports so that it would fit in the 30 page appendix.

```
!pip install gpytorch
!pip install ConfigSpace
!pip install openml
!git clone https://github.com/automl/TabPFN
tabpfn_path = 'TabPFN'
sys.path.insert(0, tabpfn_path)
num_datasets=0
pd.set_option('display.max_columns', None)
np.set_printoptions(threshold=sys.maxsize)
directory = '/content/drive/MyDrive/Colab␣Data/Organized/RunnableDatasets␣
    Ultimate/All␣Datasets␣TabPFN'
n=0
datasets = []
accuracies = []
for filename in os.listdir(directory):
```

```
f = os.path.join(directory, filename)
if os.path.isfile(f) and filename.split('.')[1]=='csv':
    print(f)
    df = pd.read_csv(f)
    df_tmp = df.drop(['Class'],axis=1)
    classifier = TabPFNClassifier(device='cuda', N_ensemble_configurations
        =32, base_path=tabpfn_path)
    cv_results_ = cross_validate(classifier, X=df_tmp.to_numpy(), y=df['
        Class'].to_numpy(), cv=3,scoring = 'roc_auc')
    tabPFN_out_df = pd.DataFrame.from_dict(cv_results_)
    out_path = '/content/drive/MyDrive/Colab␣Data/Organized/Data␣Results␣
        Ultimate/Default␣Trimmed␣TabPFN/'
    model_name = 'TabPFN'
    tabPFN_out_df.to_csv(out_path+str(filename.split('.')[0])+'/' +
        model_name+ '.csv')
```

Listing A.3: Code to run TabPFN benchmark. Disclaimer: I have trimmed many things(only affecting readability of code) and imports so that it would fit in the 30 page appendix.

defaulttabpfnthesis

Appendix B

Dataset Information

Dataset Name	**# samples**	**# features (including target)**	**Class Imbalance (Shannon Entropy)**	**Source**
salaryGr eater50k	6512/	15/	0.796/	https://www.kaggle .com/datasets/uciml /adult-census-incom e
Accelero meter_C ooler_Fan	10000/ 1500	7/4	1.0/1.0	https://archive.ics.uc i.edu/dataset/846/acc elerometer
Glioma	838/	27/	0.981/	https://archive.ics.uc i.edu/dataset/759/gli oma+grading+clinic al+and+mutation+f eatures+dataset
boston_h ouse	506/ 506	14/ 13	1/1	https://www.kaggle .com/datasets/fedes oriano/the-boston-h ouseprice-data

Raisin_ Dataset	900/ 900	8/8	1/1	https://www.kaggle .com/datasets/jsphy g/weather-dataset-r attle-package/data
amphib ians	189/	61/	0.985/	https://archive.ics.u ci.edu/dataset/528/a mphibians
Sirtuin6	100/ 100	7/7	1.0/1.0	https://archive.ics.uc i.edu/dataset/748/sirt uin6+small+molecul es-1
thryoid_ cancer	383/	17/	0.858/	https://archive.ics.uci .edu/dataset/915/diffe rentiated+thyroid+can cer+recurrence
student- mat	395/	31/	0.998/	https://www.kaggle .com/datasets/larsen 0966/student-perfor mance-data-set
Sepsis	1511/	4/	0.938/	https://archive.ics.u ci.edu/dataset/827/se psis+survival+mini mal+clinical+records
turkish_m usic_emot ion	400 / 400	51 / 51	1.0/1.0	https://archive.ics.uc i.edu/dataset/862/tur kish+music+emotion
auction_v erification	2043 /	8 /	0.553 /	https://archive.ics.uci .edu/dataset/713/auct ion+verification
car_quality	1728 /	7 /	0.881 /	https://www.kaggle. com/datasets/elikpl im/car-evaluation-d ata-set

Pumpkin_ Seeds_D ataset	2500 / 1500	13 / 13	0.999 / 1.0	https://www.muratko klu.com/datasets/
happiness	156 / 156	7/7	1.0/1.0	https://www.kaggle.co m/datasets/unsdsn/w orld-happiness/data? select=2019.csv
Gender_G ap_Spanis h_WP	706 / 706	19 / 17	1.0 /1.0	https://archive.ics.u ci.edu/dataset/852/g ender+gap+in+span ish+wp
breast can cer	569 / 569	31 / 31	0.953 / 0.953	https://archive.ics.uc i.edu/dataset/17/bre ast+cancer+wisconsi n+diagnostic
Date2nd	20 /	7 /	1.0 /	https://archive.ics.uc i.edu/dataset/879/ajw a+or+medjool
doctor_vi sits	714 /	19 /	0.688 /	https://archive.ics.uc i.edu/dataset/936/na tional+poll+on+heal thy+aging+(npha)
eng_stud ent_perfo rmance	145 /	33 /	0.996 /	https://archive.ics.uc i.edu/dataset/856/hi gher+education+stu dents+performance+ evaluation
cirrhosis	276 / 276	19/12	0.972 / 0.972	https://archive.ics.uc i.edu/dataset/878/cir rhosis+patient+surv ival+prediction+dat aset-1

day - bik e sharing	731 / 731	39 / 7	1.0 /1.0	https://www.kaggle .com/datasets/conta ctprad/bike-share-d aily-data
HCV	589 / 589	13/12	0.491 / 0.491	https://archive.ics.u ci.edu/dataset/571/h cv+data
iris	150 / 150	5/5	0.918 / 0.918	https://archive.ics.u ci.edu/dataset/53/iris
Health_N utrition_A ge	2278 / 1500	11/6	0.634 / 0.799	https://archive.ics.u ci.edu/dataset/887/n ational+health+and +nutrition+health+s urvey+2013-2014+ (nhanes)+age+predi ction+subset
CSVabalo ne	4177 / 1500	9/8	1/1	https://archive.ics.uc i.edu/dataset/1/abalo ne
Date_Frui t_Datasets	898 /898	35/35	0.999 /0.999	https://www.muratko klu.com/datasets/
LTFSID_i ntrusion_d etection	182 /	5 /	0.999 /	https://archive.ics.u ci.edu/dataset/715/ lt+fs+id+intrusion+ detection+in+wsns
maternal_h ealth_risk	1014 / 1014	7/7	0.971 / 0.971	https://archive.ics.u ci.edu/dataset/863/m aternal+health+risk
Pistachio_ 28_Featur es_Dataset	2148 / 1500	29 / 29	0.984 / 1	https://www.muratko klu.com/datasets/
Rice	3810 / 1500	8/8	0.985 / 1	https://www.muratk oklu.com/datasets/

Student_dr oput_rate_ 2nd	3630 /	239 /	0.966 /	https://archive.ics.u ci.edu/dataset/697/p redict+students+dro pout+and+academic +success
steel_energy _consumpti on	10000 / 1500	10/8	1/1	https://archive.ics.u ci.edu/dataset/851/st eel+industry+energ y+consumption
diabetes_bin ary_5050spli t_health_ind icators_BRF SS2015	10000 /	22 /	1.0 /	https://www.kaggle. com/datasets/alexte boul/diabetes-healt h-indicators-dataset /code?datasetId=17 03281&sortBy=vot eCount
parkinsons	5875 / 1500	21 / 18	0.998 / 1	https://archive.ics.uc i.edu/dataset/174/pa rkinsons
avocado	10000 / 1500	12/9	1/1	https://www.kaggle.c om/datasets/neuromu sic/avocado-prices/d ata
NY_air bnb	10000 /	12 /	1.0 /	https://www.kaggle. com/datasets/dgomo nov/new-york-city-a irbnb-open-data/cod e?datasetId=268833 &sortBy=voteCount
Cali-ho use-price	10000 / 1500	14 / 14	1/1	https://www.kaggle .com/datasets/fedes oriano/california-ho using-prices-data-ex tra-features/data

Dry_Bean_Dataset	10000 / 1500	17 / 17	1/1	https://www.muratkoklu.com/datasets/
rainAUS	10000 / 1500	22 / 17	1/1	https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package/data
NATICUSdroid	7491 /	87 /	0.934 /	https://archive.ics.uci.edu/dataset/722/naticusdroid+android+permissions+dataset
zCompanyBankruptcy	6819 / 1500	96 / 94	0.206 / 0.601	https://www.kaggle.com/datasets/fedesoriano/company-bankruptcy-prediction
PDIOT-Respeck	2394 / 1500	43 / 43	0.998 / 1	My Principles and Design of IoT Systems class created this dataset last semester
Acoustic_Extinguisher_Fire_Dataset	10000 / 1500	7 / 5	1/1	https://www.muratkoklu.com/datasets/
Mean of all 44 datasets /29 datasets	3472.2/1023.3	26.5/17.59	0.925/0.955	

Table B.1: Information for 44 datasets. #samples, #features and Class Imbalance have values of either format number/number or number/. Where the number to the right of the slash is for the trimmed TabPFN dataset and when there is no number it means the dataset was not included in the TabPFN datasets.

Appendix C

Benchmark Results

Figure C.1: AUC random search results for 44 datasets.

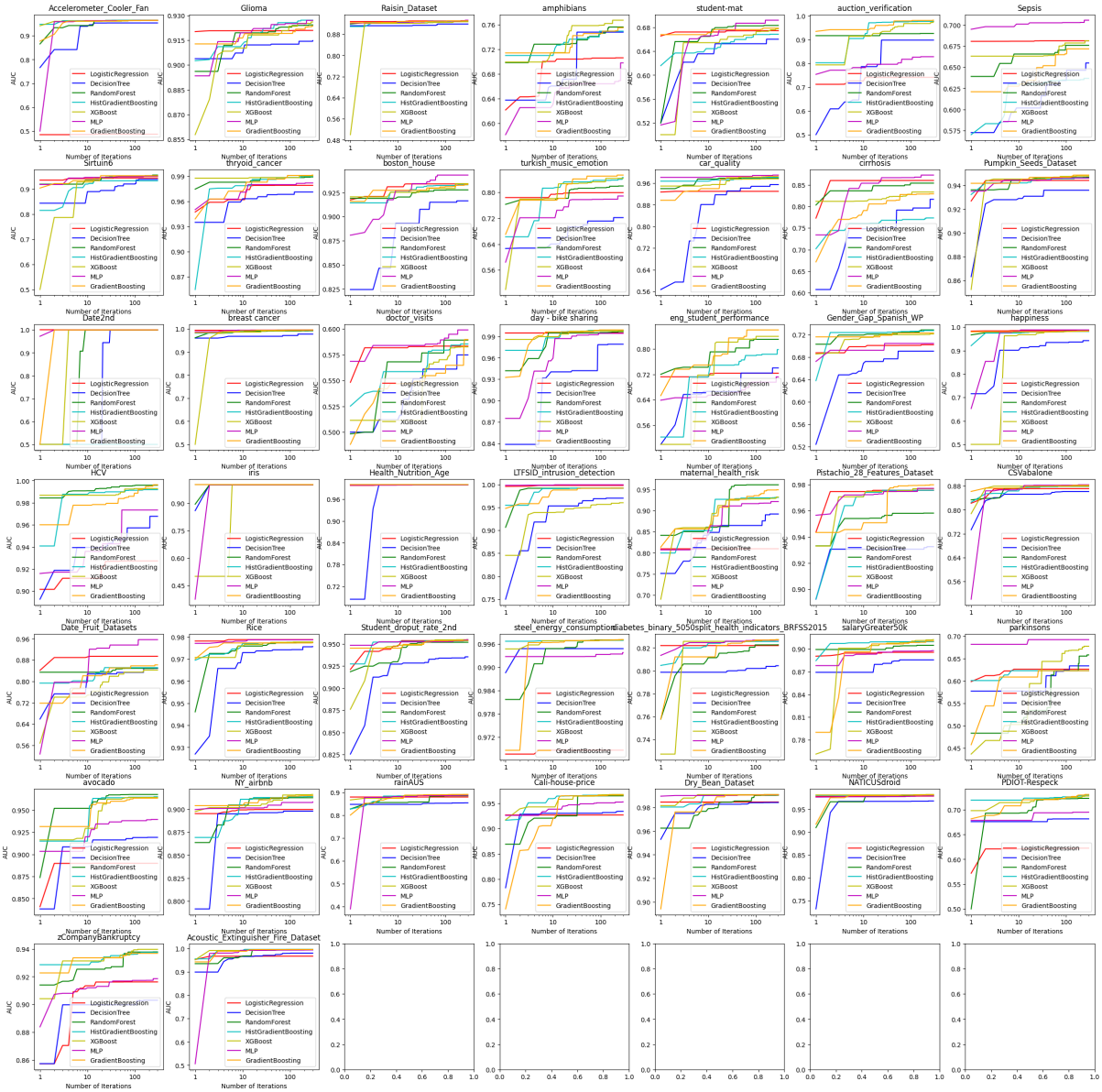

Figure C.2: AUC Bayesian optimization results for 44 datasets.

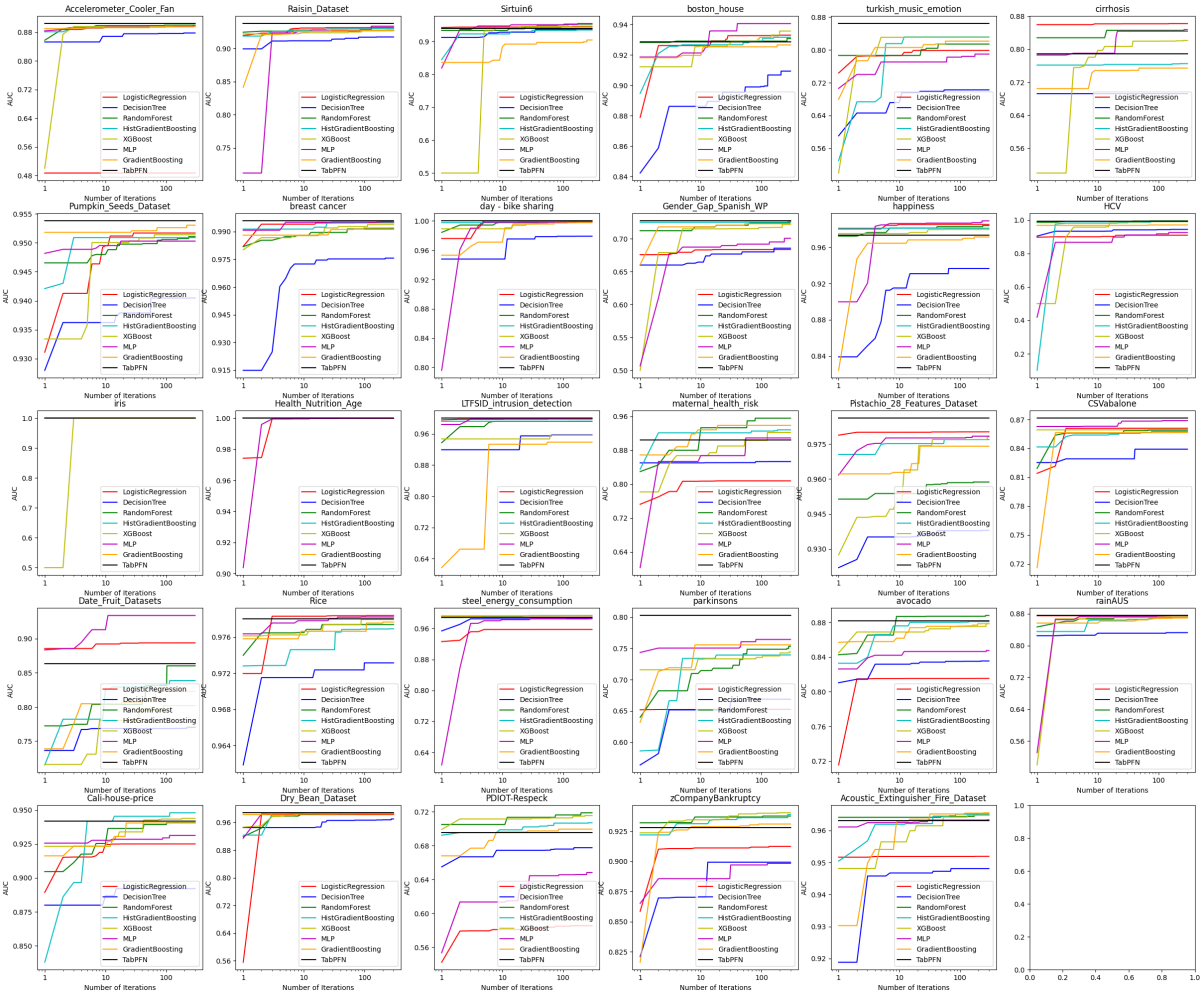

Figure C.3: AUC random search results for the 29 trimmed datasets.

Appendix D

Hyper-parameter Information

Logistic Regression	**Description**	**Distribution**
C	It is the inverse of the regularization strength	loguniform(1e-6, 1000) / Real(1e-5, 5,prior='log-uniform')
max_iter	It is the maximum number of iterations the solver is allowed to use to converge	/ Integer(50,500,prior='uniform')
fit_intercept	If a bias term should be used	/ Categorical([True,False])
penalty	Type of regularization	/ Categorical(['l2',None])

Table D.1: Logistic regression hyper-parameter distributions for random search and Bayesian optimization. Where the 'Distribution' column has the random search distribution to the left of the slash(/) and the Bayesian optimization distribution to the right of the slash. Refer to the scikit-learn library for details about the default hyper-parameters [41].

Decision Tree	Description	Distribution
ccp_alpha	It stands for cost complexity pruning. This pruning is done after the tree is trained. It compares all subtrees of the original tree and calculates a metric that defines how well fit to the training data the subtrees are. It also adds the product of the ccp_alpha with the number of leaves in the subtree. Then pick whichever subtree has the minimum. So a higher ccp_alpha lowers overfitting and a lower ccp_alpha does less pruning.	/ Real(0.0001, 0.1,prior= 'log-uniform')
max_depth	The maximum depth the decision tree is allowed to get to.	[None,2,3,4] / Integer(1,30,prior='uniform')
min_samples_leaf	The minimum number of samples that a leaf node must have.	int_loguniform(1,50) / Integer(1,50,prior='uniform')
min_samples_split	The node is only allowed to split if it has the minimum number of samples in it.	/ Integer(2,50,prior='uniform')
max_features	The subset of features allowed to be considered when splitting a node.	/ Categorical(['sqrt', 'log2', None, 0.1,0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
criterion	Measures how affective a split at a node will be.	/ Categorical(['gini','entropy'])
min_impurity_ decrease	If splitting a node results in an impurity decrease(calculated relative to the 2 child nodes) greater than min impurity decrease, it is allowed to split.	/ Categorical([0.0,0.1e-7,1e-6, 1e-5,1e-4,1e-3,1e-2])
max_leaf_nodes	The maximum number of leaves allowed. The order in which the nodes are split actually matter now as we do not want a node splitting into uninformative leaves . Hence the tree grows in best-first fashion. Where every possible node is considered and the one with the reduced impurity is split first.	int_normal(31, 5) /

Table D.2: Decision tree hyper-parameter distributions for random search and Bayesian optimization. Where the 'Distribution' column has the random search distribution to the left of the slash(/) and the Bayesian optimization distribution to the right of the slash. Refer to the scikit-learn library for details about the default hyper-parameters [42].

Random Forest	Description	Distribution
n_estimators	The number of trees that make up the ensemble.	int_loguniform(10,1000) / Integer(10, 3000, prior='log-uniform')
max_depth	The maximum depth any decision tree is allowed to get to.	[None,2,3,4] / Categorical([None,2,3,4])
min_samples_leaf	The minimum number of samples that a leaf node must have.	int_loguniform(1,50) / Integer(1, 51, prior='log-uniform')
max_features	The subset of features allowed to be considered when splitting a node.	['sqrt', 'log2', None, 0.1, 0.2, 0.3, 0.4 , 0.5, 0.6, 0.7, 0.8, 0.9] / Categorical(['sqrt', 'log2', None, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
min_impurity_ decrease	If splitting a node results in an impurity decrease(calculated relative to the 2 child nodes) greater than min impurity decrease, it is allowed to split.	/ Categorical([0.0,0.01,0.02,0.05])
bootstrap	Randomness is created through bagging(bootstrap aggregating), where each decision tree is trained using a random subset of the original training dataset.	/ Categorical([True,False])
criterion	Measures how affective a split at a node will be.	/ Categorical(['gini','entropy'])
min_samples_split	The node is only allowed to split if it has the minimum number of samples in it.	/ Categorical([2,3])

Table D.3: Random forest hyper-parameter distributions for random search and Bayesian optimization. Where the 'Distribution' column has the random search distribution to the left of the slash(/) and the Bayesian optimization distribution to the right of the slash. Refer to the scikit-learn library for details about the default hyper-parameters [43].

Gradient Boosting	Description	Distribution
learning_rate	Also known as the shrinkage. A higher learning_rate means that the decision trees have more influence and vice versa.	lognorm(scale = (0.01), s=np.log(10)) / Real(1e-6, 0.3, prior='log-uniform')
min_samples_leaf	The minimum number of samples a leaf node must have.	/ Integer(1,50,prior='log-uniform')
max_depth	The maximum depth any decision tree is allowed to get to.	[None,2,3,4,5] / Categorical([None,2,3,4,5])
loss	The loss function to be optimized.	/Categorical(['deviance', 'exponential'])
subsample	Similar to random forest bagging. The subset of samples each decision tree is allowed to use.	/ Real(0.5,1,prior='uniform')
n_estimators	The number of trees that make up the ensemble.	int_loguniform(10,500)# / Integer(10,1000, prior='log-uniform')
criterion	Measures how affective a split at a node will be.	/ Categorical(['friedman_mse', 'squared_error'])
min_samples_split	A node is only allowed to split if it has minimum number of samples in it.	/ Integer(2,3,prior='uniform')
max_leaf_nodes	The maximum number of leaves allowed.	/ Categorical([None,5,10,15])
min_impurity_decrease	If splitting a node results in an impurity decrease(calculated relative to the 2 child nodes) greater than min impurity decrease, it is allowed to split.	/ Categorical([0.0,0.01,0.02,0.05])

Table D.4: Gradient boosting hyper-parameter distributions for random search and Bayesian optimization. Where the 'Distribution' column has the random search distribution to the left of the slash(/) and the Bayesian optimization distribution to the right of the slash. Refer to the scikit-learn library for details about the default hyper-parameters [44].

HistGradient Boosting	**Description**	**Distribution**
learning_rate	Also known as the shrinkage. A higher learning_rate means that the decision trees have more influence and vice versa.	lognorm(scale = (0.01), s=np.log(10)) / Real(1e-6, 0.3,prior= 'log-uniform')
min_samples_leaf	The minimum number of samples a leaf node must have.	int_normal(20,2) / Integer(16, 24, prior='uniform')
max_leaf_nodes	The maximum number of leaves allowed.	int_normal(31, 5) / Integer(22, 40, prior='uniform')
max_depth	The maximum depth any decision tree is allowed to get to.	[None,2,3,4] / Categorical([None,2,3,4])
l2_regularization	Controls the influence of the l2 regularization.	/ Real(1e-5,0.1, prior='log-uniform')
max_iter	Somewhat equivalent to n_estimators in other boosting algorithms. It is the maximum number of iterations allowed for the boosting procedure. It differs to n_estimators in that there is no gaurentee of having a max_iter amount of trees as it might stop early.	/ Integer(50,200, prior='uniform')
max_bins	The max number of bins allowed when discretizing the input. It can never be greater than 255 and one bin is always set aside for missing values.	/ Integer(100,255, prior='uniform')

Table D.5: Histogram gradient boosting hyper-parameter distributions for random search and Bayesian optimization. Where the 'Distribution' column has the random search distribution to the left of the slash(/) and the Bayesian optimization distribution to the right of the slash. Refer to the scikit-learn library for details about the default hyper-parameters [45].

XGBoost	Description	Distribution
learning_rate	Also known as the shrinkage. A higher learning_rate means that the decision trees have more influence and vice versa.	loguniform(1e-5, 0.7) /Real(1e-5, 0.7, prior='log-uniform')
n_estimators	The number of trees used.	int_loguniform(90,1000) /Integer(90, 1000, prior='log-uniform')
max_depth	The maximum depth any tree is allowed to get to.	list(range(1,11)) /Integer(1, 11, prior='uniform')
min_child_weight	A node is allowed to split if the sum of the child nodes instance weight is greater than or equal to min_child_weight.	int_loguniform(1,1e2) /Integer(1, 1e2, prior='log-uniform')
reg_alpha	Controls the influence of L1 regularization.	loguniform(1e-8, 1e2) /Real(1e-8, 1e2, prior='log-uniform')
colsample_bytree	Random subset of features to be used for each tree	uniform(0.5,0.5) /Real(0.5,1, prior='uniform')
subsample	Similar to random forest bagging. The subset of samples each decision tree is allowed to use.	uniform(0.5,0.5) /Real(0.5, 1, prior='uniform')
predictor	Algorithm used to make predictions. Necessary if you want to use the cloud GPU's.	Always 'gpu_predictor' / Always 'gpu_predictor'
tree method	Algorithm used to train the trees. Necessary if you want to use the cloud GPU's.	Always 'gpu_hist'/ Always 'gpu_hist'
gamma	To split a node the reduction in loss must at least be greater than gamma.	/Real(1e-8, 7, prior='log-uniform')
reg_lambda	Controls influence of L2 regularization.	/Real(1, 4, prior='log-uniform')
colsample_bylevel	Random subset of features to be used for each level of every tree	/Real(0.5,1, prior='uniform')

Table D.6: XGBoost hyper-parameter distributions for random search and Bayesian optimization. Where the 'Distribution' column has the random search distribution to the left of the slash(/) and the Bayesian optimization distribution to the right of the slash. Refer to the scikit-learn library for details about the default hyper-parameters [46].

MLP	Description	Distribution
num_hidden_layers	The number of hidden layers .	[1,2,3] / Integer(1,5)
Hidden layer sizes	The number of nodes in the hidden layers.	Layer 1,2 and 3: list(range(10, 251)) /Layer 1,2 and 3: Integer(1,200)
weight_decay	Controls L2 regularization.	loguniform(1e-5, 0.1) /Real(1e-6, 1, prior='log-uniform')
Learning rate	Controls how much the weights and bias get updated per batch.	loguniform(1e-6, 1e-2) /Real(1e-5, 1e-2, prior='log-uniform')
Batch size	Number of training samples used to update the weights and bias when optimizing.	list(range(64, 513)) /Integer(64, 512) Unfortunately, I did not use powers of 2. This did not result in lower AUC, however it did result in a (negligible) increase in training time. This is due to memory alignment issues.
Max_epochs	The maximum number of 'cycles' through a dataset that is allowed when optimizing	/Integer(50,500)
eps	Used for stability in the optimizer to avoid division by zero.	/ Real(1e-10, 1e-6, prior='log-uniform')
nonlin	Activation function to use.	/Categorical([nn.Identity(), nn.Sigmoid(), nn.Tanh(),nn.ReLU()])

Table D.7: MLP hyper-parameter distributions for random search and Bayesian optimization. Where the 'Distribution' column has the random search distribution to the left of the slash(/) and the Bayesian optimization distribution to the right of the slash. Refer to the code snippets A.1 and A.2 for details of the fixed hyper-parameter values for random search and Bayesian optimization respectively.